IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 5 June 2024, accepted 16 June 2024, date of publication 19 June 2024, date of current version 17 July 2024.

*Digital Object Identifier 10.1109/ACCESS.2024.3416812*

RESEARCH ARTICLE

# Hardware Acceleration of Crystals-Kyber in Low-Complexity Embedded Systems With RISC-V Instruction Set Extensions

**CARLOS GEWEHR, LUCAS LUZA, AND FERNANDO GEHM MORAES, (Senior Member, IEEE)**
School of Technology, Pontifical Catholic University of Rio Grande do Sul (PUCRS), Porto Alegre 90619-900, Brazil

Corresponding author: Carlos Gewehr (carlos.gewehr@edu.pucrs.br)

This work was supported in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico–CNPq under Grant 309605/2020-2 and Grant 407829/2022-9, and in part by the Fundação de Amparo à Pesquisa do Estado do RS–FAPERGS under Grant 21/2551-0002047-4 and Grant 23/2551-0002200-1.

**ABSTRACT** The imminent rise of practical quantum computing threatens well-established cryptography algorithms for secret key exchange in use today, such as Diffie-Hellman, RSA and Elliptic Curve based schemes. To answer this challenge, the National Institute for Standard and Technology (NIST) has launched a competition for Key Encapsulation Mechanism (KEM) algorithms showing resistance to classical and quantum-based attacks. In July 2022, NIST announced that the Crystals-Kyber algorithm was chosen as the competition's winner, being standardized as ML-KEM. This work aims to explore hardware acceleration through Instruction Set Extensions (ISEs) in a low-end 32-bit RISC-V core (Ibex) in a comprehensive evaluation of performance, energy consumption, memory footprint and die area costs. Four different parametrizations of Kyber symmetric primitives are evaluated: the well-known SHA-3 and AES/SHA-2 based versions (Kyber-FIPS202 and Kyber-90s); and 2 novel parametrizations using Ascon and TurboSHAKE (Kyber-Ascon and Kyber-Turbo). Hardware acceleration of symmetric primitives in Kyber-90s with the *Zkne* and *Zknh* ISEs shows performance and energy gains of 42% and 37% for Kyber-512 encapsulation. Combining the acceleration of primitives with a novel *Xkyber* ISE accelerating polynomial arithmetic, CBD sampling and coefficient compression, further gains of 47% and 41% in performance and energy consumption are observed (for Kyber-512 encapsulation), while also reducing Kyber code size by 15%. *Xkyber* area costs are of 13% of the baseline Ibex processor with no ISEs or 4.3K equivalent gates. Software and RTL implementations are publicly available at github.com/cggewehr/RISCV-crypto.

**INDEX TERMS** Post-quantum cryptography, Crystals-Kyber, embedded systems, low power, RISC-V.

## I. INTRODUCTION

The advancement of quantum computing has raised concerns about the long-term security of current schemes for establishing keys for symmetric cryptography, such as Elliptic Curve Diffie-Hellman (ECDH), now known to be vulnerable to quantum based attacks [1]. Novel algorithms based on mathematical foundations resistant to such attacks are needed, introducing a new research field called Post-Quantum Cryptography (PQC).

Addressing this need, the National Institute of Standards and Technology (NIST) promoted a competition to standardize a new post-quantum key exchange algorithm. The winning algorithm was announced to be Crystals-Kyber [2], which will be standardized under the name ML-KEM [3]. Key establishment via Kyber introduces significant performance and memory usage overheads, increasingly relevant in Internet of Things (IoT) devices with limited resources, which usually forward data to complex upstream nodes

The associate editor coordinating the review of this manuscript and approving it for publication was Chien-Ming Chen.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

for processing. Due to this emerging computing paradigm (edge computing), the efficient provision of quantum-secure communication in low-complexity endpoints is key for the further advancement of the IoT.

Parallel to the PQC competition, NIST has also promoted the Lightweight Cryptography (LWC) competition, aiming to standardize algorithms with authenticated encryption and optionally hashing capabilities especially tailored to resource-constrained devices. In February 2023, the Ascon [4] algorithm family was selected as the winner of the LWC competition. The intersection between LWC and PQC remains an open problem.

Hardware acceleration can significantly enhance the performance and energy efficiency of an algorithm [5]. One way to achieve hardware acceleration is by implementing specialized instructions in a general-purpose processor, a practice known as Instruction Set Extensions (ISEs). Compared to memory-mapped accelerators, extending a base instruction set with specialized instructions provides benefits relevant to low-power embedded systems, including: (*i*) resource sharing among generic and specialized components, such as the register file and RAM interfaces; (*ii*) trivial data transfer among generic and specialized hardware via the register file, avoiding costly memory accesses; (*iii*) no added system complexity to e.g. the bus and interrupt controllers.

This work details the design and evaluation of 4 parametrizations of the symmetric primitives in Crystals-Kyber in the context of the low-complexity Ibex RISC-V core [6], considering both software-only and hardware-accelerated implementations of each Kyber parametrization. Our quantitative analysis assesses the effectiveness of these implementations across several metrics critical to resource-constrained embedded systems. These metrics include performance, software code size, memory footprint, hardware die area usage, and energy consumption.

The original contributions of this work include:

- an evaluation of the use of Ascon and TurboSHAKE as symmetric primitives within Crystals-Kyber;
- a comparative analysis of hardware acceleration via ISEs of 4 different symmetric primitive parametrizations of Kyber, including the previously proposed Keccak-based and "Kyber-90s" versions as well as the aforementioned parametrizations using Ascon [4] and TurboSHAKE [7];
- the proposal of the novel *Xkyber* ISE, comprising polynomial arithmetic operations, CBD (Centered Binomial Distribution) sampling and coefficient compression.

The remainder of this document is organized as follows. Section II reviews relevant literature and positions this work within the state-of-the-art. Section III concisely introduces the Crystals-Kyber algorithm. Section IV details each of the 4 symmetric parametrizations of Kyber explored in this work and how they are accelerated with ISEs. Section V introduces the novel *Xkyber* extension and describes its implementation in the Ibex processor. Section VI presents the experimental evaluation of the four symmetric parametrizations and the *Xkyber* extension. Finally, Section VII summarizes the conclusions drawn from the experimental evaluation and outlines directions for future research.

## II. RELATED WORK

In this section, we review and summarize relevant works in the literature that discuss RISC-V ISEs proposals for symmetric cryptography algorithms being used as primitives within Kyber, as detailed in Section II-A. A similar analysis is conducted for ISE proposals specifically related to Crystals-Kyber, presented in for symmetric cryptography algorithms being used as primitives within Kyber, as detailed in Section II-B.

### A. SYMMETRIC CRYPTOGRAPHY ISES

Bertoni et al. [8] present several optimization techniques for the implementation of the Keccak permutation. Stoffelen [9] evaluates the SHA-3 optimization techniques presented in [8] in an assembly implementation focused on performance rather than on code size. The Authors report 13,774 clock cycles for the Keccak permutation in a SiFive E31 core.

Campos et al. [10] provide another similar software-only evaluation of the Keccak permutation, porting the implementation from [9] to C. An improvement of 3% due only to using C instead of assembly is reported. Additionally, the Authors evaluate the Keccak permutation considering the bit manipulation (*B*) extension performed on the Spike simulator. Results of 12,402 and 14,633 clock cycles are reported, with and without the bit manipulation extension, respectively. The Authors also consider Ascon in their evaluation, considering an inline ASM implementation of the Ascon permutation, and one permutation using the bit-interleaving technique, both using loop unrolling.

Saarinen [11] implements the hash functions and XOFs on the SHA-3 standard. This implementation is more appropriate to the resource-constrained context being explored, as it does not make use of loop-unrolling such as in previous implementations in [9] and [10], while still being optimized in light of the techniques presented in [8]. It can also be configured to use the *Zbkb* extension.

Software-only evaluations concerning the entire RISC-V cryptography ISE are reported in [12] and [13]. Specifically, Nianc et al. [12] report clock cycle count reductions of 43% for SHA-256 and 40% for SHA-512, along with decreases in program memory usage of 33% and 32%, respectively. Similar results are reported in the earlier Marshall et al. [13] work, evaluated in the SCARV [14] core.

Cheng et al. [15] propose ISEs for several LWC algorithms, among them Ascon. In the case of Ascon, they propose a new custom extension called *Xascon* and evaluate it considering software-only and *Zbkb*-accelerated implementations. The evaluation for the *Xascon* extensions makes use of both *Xascon* and *Zbkb*. The *Xascon* + *Zbkb* implementation shows 2.28x and 1.18x performance gains compared to

software-only and *Zbkb*-only implementations, respectively. Authors perform their work in the context of the Rocket core [16], with 5 pipeline stages and floating point instructions, more complex than expected for an IoT context. Hardware-specific metrics are evaluated in FPGAs but not ASIC implementations. The *Xascon* + *Zbkb* and *Zbkb*-only extended Rocket RV32GC processors come at an area cost of 1.282x and 1.140×, respectively, measured in LUTs used.

Marshall et al. [17] present a comparative analysis of several RISC-V ISE proposals, where the Saarinen proposal [18] is demonstrated to be the best AES ISE evaluated, which was later standardized as the *Zkne* and *Zknd* extensions in the RISC-V scalar cryptography. Synthesis results use the open-source Yosys tool. In the SCARV [14] core, this ISE shows a 3% area increase, accelerating both AES encryption and decryption. Saarinen [18] reports a 5% increase in LUT usage in an FPGA-based evaluation considering a lower complexity single-cycle RISC-V core. No power or energy results are provided. In the software side of the evaluation in [17], the [18] proposal shows 3.5x and 1.65x performance gains for AES-128 encryption and encryption key expansion, respectively. The software comparisons consider a higher-performance T-Table AES implementation with 4 KB LUTs, trading off memory usage for performance.

In our previous work [19], we explored the *Zkne* and *Zknh* extensions for AES encryption and SHA-2 hashing, respectively. Within the Ibex core framework, both extensions introduced an area overhead of approximately 10% while achieving performance gains exceeding 42x for AES encryption and an average of 1.6x for SHA-2 hashing. In a subsequent work [20], we evaluated authenticated encryption using AES-128 in CCM mode, Ascon-128, and ChaCha20-Poly1305 algorithms.

### B. KYBER-SPECIFIC ISES

Albrecht et al. [21] explore the use of loosely coupled SHA-2 and AES accelerators in a smart-card SoC (SLE 78) in the Kyber-90s version of Kyber. Conversely, Park et al. [22] explore the use of loosely coupled RSA/ECC accelerators to speed up polynomial arithmetic. This is done through the Kronecker substitution technique, used to compute polynomial multiplications as regular integer multiplications, which can be accelerated on existing RSA/ECC hardware.

Banerjee et al. [23] present a generic accelerator for MLWE algorithms, implementing operations such as modular multiplication for a range of prime moduli and various random samplers from a SHA-3 PRNG. This accelerator integrates several intermediary operations, yielding great performance improvements, but incurring in a significant area overhead of 106K equivalent gates. Thus, it is not an appropriate solution for the embedded context, similar to hardware-only solutions.

Xin et al. [24] propose a RISC-V vector co-processor for lattice-based post-quantum cryptography. The co-processor implements usual operations needed in lattice-based algorithms, such as coefficient sampling integrated into a Keccak-1600 core, parallel butterflies for the Number Theoretic Transform (NTT), as well as modular arithmetic. This being a vector implementation, parallelism is explored extensively, not only in arithmetic operation execution itself but in buffering relevant inputs, such as the output of the Keccak PRNG in a FIFO and twiddle factors for the NTT in a local RAM. This implementation offers a significant performance increase, but is very expensive in area, making it not suitable for the low-resource context being explored.

Fritzmann et al. [25] propose two accelerators for hash (Keccak) and NTT operations coupled to a main processor via an AHB bus. Subsequent enhancements are discussed in [26], where the Authors integrate the aforementioned co-processors inside a general-purpose CV32E40P processor. Their approach requires the presence of a floating point register file for the integration of the Keccak core, leading to significant area overheads in a higher complexity processor than the Ibex processor. Furthermore, the vectorized polynomial arithmetic component is implemented independently of the existing integer arithmetic ALU in the base processor, not exploring resource sharing.

Nannipieri et al. [27] show a superscalar 6-stage 64-bit CVA6 RISC-V processor extended with custom instructions for the Kyber and Dilhitium algorithms. These instructions are implemented based on a separate functional block, named PQ-ALU. The PQ-ALU implements the usual modular arithmetic operations, as well as a one-cycle CT butterfly. This approach is not usable in a low-resource context due to a lack of resource sharing, both within the PQ-ALU and among existing arithmetic elements in the base processor. There are 5 16-bit multipliers in a combinational path within the PQ-ALU, which, surprisingly, are reported to not be in the critical path of the extended processor. This approach is reasonable considering the context of the CVA6 64-bit superscalar processor, but not for the low-complexity Ibex core.

Lee et al. [28] present a co-processor based approach, with the usual operations needed for Kyber and other MLWE-based algorithms, such as Keccak, sampling, and polynomial multiplication, as well as the standardized RISC-V scalar cryptography extensions, accelerating AES and SHA-2. No quantitative results are presented, but we can conclude that this approach is not optimal, considering that the arithmetic modules are separated from the processor integer ALU, and intermediary results are committed to a separate register file in the co-processor. Again, resource sharing is not explored to the extent necessary for IoT applications.

In Alkim et al. [29], hardware-software co-design is considered in the context of a 32-bit 5-stage RISC-V processor. New instructions for optimizing the NTT computation are implemented such that twiddle factors can be generated locally instead of being fetched from a pre-computed table stored in memory, as usually done in software implementations, reducing code size and the number of memory accesses.

However, resource sharing is not explored. In the base processor (VexRiscv) used in this proposal, regular integer multiplication computations are distributed between pipeline stages of the processor, and the same structure is followed for modular multiplication and reductions. Furthermore, the Authors implement the modular multiplication and reduction completely independently of the integer multiplication units in the base processor, such that the processor can be instanced without the integer multiplier while the optimizations proposed by the authors still apply. Again, this is not an optimal use of resource sharing between regular integer arithmetic elements and modular arithmetic elements.

Karabulut et al. [30] present a different approach to optimizing the NTT. Instead of implementing new instructions as in previous works, the Authors propose modifications to the base processor control signals, such that memory dependence prediction and out-of-order execution are accomplished when purely software butterflies are detected during code execution. Again, this proposal is made in the context of a complex processor and does not apply to resource-constrained devices.

Miteloudi et al. [31] present a custom extension for polynomial arithmetic, supporting the Kyber and Dilithium algorithms. The supported operations are modular addition, subtraction, and multiplication, as well as CT and GS butterflies, implemented in a separate ALU decoupled from the regular integer arithmetic elements in the RI5CY core. As in previous proposals, the Authors only consider the acceleration of modular arithmetic. The single-cycle butterfly computations consider a register file with two write ports, which is highly unusual and not expected to be supported in most low-complexity RISC-V implementations.

Summarizing, we can see that none of the works reviewed sufficiently address the issue of PQC and the Kyber algorithm in resource-constrained environments considering ISEs. Previously proposed solutions are either of high complexity or integrated inside complex processors. An implementation that explores extensive resource sharing with existing integer arithmetic elements in a naturally low-complexity processor is still missing in the literature.

Also, previous works often overlook the evaluation of energy consumption and memory footprint for their proposed solutions. Our work addresses this gap by presenting comprehensive analyses of both factors, offering a deeper evaluation of the implications of hardware acceleration through ISEs. Additionally, reported area overhead values are based on open-source synthesis tools or FPGA implementations, which may not accurately reflect the metrics in practical ASIC implementations. In contrast, this work employs a commercial synthesis tool with a manufacturable cell library, ensuring high-quality area results.

Table 1 summarizes characteristics the works reviewed, concerning their hardware acceleration of symmetric primitives in Kyber, polynomial arithmetic, lightweight SIMD, CBD sampling, and coefficient compression features.

**TABLE 1. Related works comparison.**

| Work | Sym. Prim. | Poly. Ops. | LW SIMD | CBD | Compress |
|---|---|---|---|---|---|
| Albrecht et al. [21] | X | X | | | |
| Park et al. [22] | | X | | | |
| Banerjee et al. [23] | X | X | | | |
| Xin et al. [24] | | X | | X | |
| Fritzmann et al. [26] | X | X | | X | |
| Nannipieri et al. [27] | | X | | | |
| Lee et al. [28] | | X | | X | |
| Alkim et al. [29] | | X | | | |
| Karabulut et al. [30] | | X | | | |
| Miteloudi et al. [31] | | X | | | |
| **This work** | X | X | X | X | X |

## III. A BRIEF INTRODUCTION TO CRYSTALS-KYBER

Among the various mathematical problems suitable for constructing quantum-secure cryptographic schemes, those involving lattices are of particular interest. A $n$-dimensional lattice can be understood as a regular discrete subset of a $n$-dimensional vector space. Like vector spaces, lattices are defined by $n$ linearly independent $n$-dimensional vectors. Such a set of vectors is called a basis of the lattice. It is common to consider only integer lattices, in which the basis and all members of the lattices have integer coefficients. Furthermore, coefficients $c_i$ are usually restricted to a $0 \leq c_i < q$ subset of the integers, where $q$ is determined by the specific algorithm. Operations involving coefficients are always performed modulo $q$ for each coefficient.

Formally, let $\mathbf{v} \in \mathbb{Z}_q^n$ to be a $n$-dimensional vector whose coefficients $c_i \in \{0, 1, \ldots, q-1\}$ and $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$ a matrix of $n$ vectors $\in \mathbb{Z}_q^n$. Throughout this text, matrices will be represented by uppercase bold letters while vectors will be represented by lowercase bold letters.

One of the lattice problems of interest to cryptography is the Closest Vector Problem (CVP): given a basis for a lattice and a point in the vector space of the same dimensions, the goal is to find the point in the lattice that is closest to the given point in the vector space. Despite showing resistance to classical and quantum-based attacks, CVP has no immediate applications in cryptography, but is crucial to the understanding of higher complexity problems that are used in practical applications, such as LWE.

The LWE problem closely resembles asymmetric cryptography, seeing as it is integral to the problem definition that some pieces of data are public while others are private: knowing a public basis matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$ and a vector $\mathbf{t} \in \mathbb{Z}_q^n$, the goal is to find a secret $\mathbf{s} \in \mathbb{Z}_q^n$ such that $\mathbf{A}s + \mathbf{e} = \mathbf{t}$, where $\mathbf{e}$ is sampled from an error distribution. Note that the LWE problem can be interpreted as forcing a CVP in a lattice of choice: $\mathbf{A}s$ gives a point in the lattice defined by $\mathbf{A}$. When $\mathbf{A}s$ is added to the error distribution $\mathbf{e}$, this offsets the point in the lattice by a small amount, such that, via the CVP, it is hard to obtain the point $\mathbf{A}s$ knowing only $\mathbf{t}$ and $\mathbf{A}$.

Note that the LWE problem operates on the algebraic structure of integers. From the implementation point of view, this is somewhat undesirable, considering a multiplication

between two vectors (dot product) does not yield another vector, but an integer. This issue can be solved if polynomials are used instead of vectors of integers, where the result of a multiplication between two polynomials yields another polynomial, not an integer.

The Module Learning With Errors (MLWE) problem is a variant of the standard LWE problem in which polynomials are used instead of integer vectors. In MLWE the $\mathbf{A}$ matrix is composed of $k \times k$ members of the $\mathbb{Z}_q[X]/(X^n + 1)$ polynomial ring i.e. the polynomials of degree less than $n$ with coefficients $c_i \in \{0, 1, \ldots, q-1\}$, while the $\mathbf{t}$ and $\mathbf{s}$ vectors are composed of $k$ members of $\mathbb{Z}_q[X]/(X^n + 1)$.

Crystals-Kyber [2] is a Key Encapsulation Mechanism (KEM), a method for establishing secret keys for symmetric cryptography. Based on the MLWE problem, it shows resistance against classical and quantum attacks. Kyber has a fixed plaintext size of 256 bits, allowing for securely establishing a key for block ciphers as strong as AES-256, which is the highest NIST security level for the AES block cipher. Kyber has 3 defined security levels, allowing for a faster and more compact implementation if the highest security level is not required for a specific scenario, i.e. using AES-128 (with Kyber-512) or AES-192 (with Kyber-768).

Table 2 shows parameters for each Kyber security level. $n$, $k$ and $q$ set the polynomial ring of the form $\mathbb{Z}_q[X]/(X^n + 1)$ for the MLWE problem as $\mathbb{Z}_{3329}[X]/(X^{256} + 1)$ and the size for matrices and vectors of polynomials in the same ring. $\eta_1$, $\eta_2$, $d_u$ and $d_v$ specify the error distribution width ($\eta_1$, $\eta_2$) and ciphertext compression factors ($d_u$, $d_v$), balancing security, ciphertext size and decryption failure probability ($\delta$).

**TABLE 2. Kyber parameters [2].**

| Kyber version | $n$ | $k$ | $q$ | $\eta_1$ | $\eta_2$ | $(d_u, d_v)$ | $\delta$ |
|---|---|---|---|---|---|---|---|
| Kyber-512 | 256 | 2 | 3329 | 3 | 2 | $(10, 4)$ | $2^{-139}$ |
| Kyber-768 | 256 | 3 | 3329 | 2 | 2 | $(10, 4)$ | $2^{-164}$ |
| Kyber-1024 | 256 | 4 | 3329 | 2 | 2 | $(11, 5)$ | $2^{-174}$ |

Like any KEM, Kyber comprises 3 distinct steps: `KeyGen`, `Enc` and `Dec`, respectively the public and private key generation; encryption of plaintext; and decryption of ciphertext. Key generation can be understood directly from the MLWE problem statement, with the vector $\mathbf{t}$ and matrix $\mathbf{A}$ used as the public key and the vector $\mathbf{s}$ used as the private key. The coefficients of each polynomial in the $\mathbf{A}$ matrix are sampled at random from $\mathbb{Z}_q$, with equal probability for each possible coefficient. Coefficients in $\mathbf{s}$ and $\mathbf{e}$ are sampled from a Centered Binomial Distribution (CBD), parameterized by the $\eta_1$ parameter from Table 2. $CBD_\eta$ returns a random element in the $-\eta, \ldots, 0, \ldots, \eta$ range computed from $2\eta$ pseudo-random input bits, as shown in Algorithm 1.

In the `Enc` step, a new value $\mathbf{r}$ is sampled from $\text{CBD}_{\eta_1}$, analogous to $\mathbf{s}$ in the `KeyGen` step. $\mathbf{r}$ is used to compute two values $\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$ and $v = \mathbf{t}^T\mathbf{r} + e_2 + m'$ are generated from the public key ($\mathbf{A}$ and $\mathbf{t}$) and scaled plaintext ($m'$), where $\mathbf{e}_1$ and $e_2$ are sampled from $\text{CBD}_{\eta_2}$. Each bit in the plaintext

**Algorithm 1** $CBD_\eta$ Sampling

**Require:** *B is an array of* $2\eta$ *pseudo-random bits*
**Ensure:** *Returns a pseudo-random element* $\in \{0, 1, \ldots, \eta, q-\eta-1, \ldots, q-1\}$
$a \leftarrow \sum_{i=0}^{\eta-1} B_i$ ▷ Sum $\eta$ lower bits of $B$
$b \leftarrow \sum_{i=\eta}^{2\eta-1} B_i$ ▷ Sum $\eta$ upper bits of $B$
**return** $a - b \bmod q$

is scaled as to be a coefficient in $\mathbb{Z}_q$: If a plaintext bit is 1, it is scaled up to $q/2$, else, it remains as $0 \in \mathbb{Z}_q$.

The ciphertext $(\mathbf{u}, v)$ is compressed according to the $(d_u, d_v)$ parameters in Table 2, reducing ciphertext size and introducing additional error due to lossy compression. The compression algorithm is shown in Algorithm 2, where a naturally 12 bit coefficient, is compressed to a $d$ bit integer.

**Algorithm 2** $Compress_d$ Kyber Coefficient Compression

**Require:** *x is a polynomial coefficient* $\in \mathbb{Z}_q$
**Ensure:** *Returns an element* $\in \mathbb{Z}_{2^d}$
**return** $\lceil (2^d/q) \cdot x \rfloor \bmod 2^d$

Where $\mathbf{u}$ and $v$ are used in the `Dec` step to obtain $m'' = v - \mathbf{s}^T\mathbf{u}$. At this point, $m''$ contains the scaled plaintext $m'$ plus several small errors terms and the error due to compression, when all summed up are less than $q/2$. The actual plaintext $m$ is recovered from $m''$ by performing a reverse scaling process, where a coefficient $\in \mathbb{Z}_q$ is mapped to 1 if greater than $q/2$, and mapped to 0 if less than $q/2$.

Note that a significant portion of the intermediary operations in Kyber are polynomial multiplications. It is worth noting that the time complexity of the trivial algorithm for multiplying two $n$-degree polynomials is $O(n^2)$. However, by using the Number Theoretic Transform (NTT), it is possible to reduce the time complexity to $O(n\ log_n)$.

Polynomial multiplication ($\mathbf{c} = \mathbf{ab} | \mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_q[X]/(X^n + 1)$) via the NTT works by performing a forward transform on the two polynomials of interest ($\hat{\mathbf{a}} = NTT(\mathbf{a})$, $\hat{\mathbf{b}} = NTT(\mathbf{b})$), pairwise multiplying the coefficients of the two transformed polynomials ($\hat{\mathbf{c}}_i = \hat{\mathbf{a}}_i\hat{\mathbf{b}}_i\ mod\ q$), then taking the inverse transform (INTT) of the pairwise multiplication ($\mathbf{c} = INTT(\hat{\mathbf{c}})$). Concisely, $\mathbf{c} = \mathbf{ab} = INTT(NTT(\mathbf{a}) \cdot NTT(\mathbf{b}))$, where "$\cdot$" denotes pairwise multiplication *mod* $q$.

A simplified version of the NTT with $n = 8$ can be visualized in Figure 1. Note that there are $log_n$ layers, in each $n$ operations are performed, giving the previously mentioned $O(n\ log_n)$ time complexity. The operation in question for the forward transform is called the Cooley-Tukey (CT) butterfly, which takes in two coefficients and pre-computed constants called "twiddle factors". The inverse transform follows the same structure, but performing the inverse base operation, called the Gentleman-Sande (GS) butterfly. The CT and GS butterflies are depicted in Figure 2.

The reader may notice a similarity of the process shown in Figure 1 to a standard Fast Fourier Transform (FFT). This

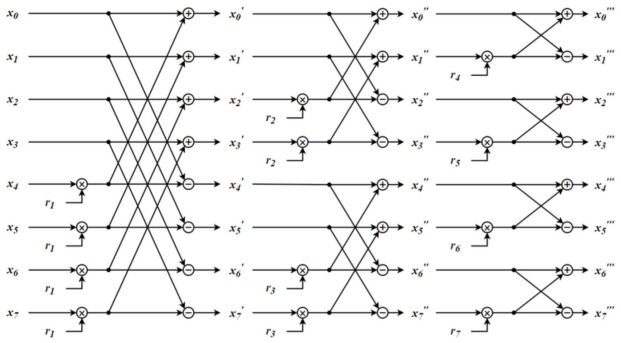


**FIGURE 1.** NTT example for $n = 8$ [32].

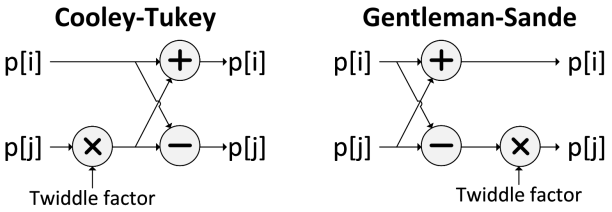



**FIGURE 2.** Cooley-Tukey (CT) & Gentleman-Sande (GS) butterflies [27].

is not a coincidence. The NTT can be understood crudely as an FFT that considers arithmetic in a $\mathbb{Z}_q$ ring instead of the complex numbers as in the usual FFT. For a deeper conceptual exposition on the NTT, as well as a comprehensive review of implementation techniques, the reader should refer to [33].

This constitutes the base Kyber algorithm, in the round-3 Kyber submission called Kyber.CPAPKE, constructed as to resist a Chosen Plaintext Attack (CPA). From this, a Chosen Ciphertext Adaptive Attack (CCA2) – secure scheme called Kyber.CCAKEM is constructed via the Fujisaki-Okamoto (FO) transform [2]. This provides a second parametrization of Kyber, in which the hashes for the FO transform and pseudo-random functions for random sampling are chosen as either the hash functions and XOFs from the FIPS-202 standard or hashes from the SHA-2 standard and AES-256 in CTR mode as a PRNG, respectively. The version of Kyber in which the FIPS-202 [34] standard is used is referred to as "Kyber-FIPS202", while "Kyber-90s" uses SHA-2 [35] and AES [36]. A third parametrization of Kyber called "Kyber-Ascon" is proposed in this work, using the Ascon [4] family of algorithms for the same functionalities. Seeing as Ascon only provides primitives at the 128 bit security level, Kyber-Ascon is evaluated only for Kyber-512. Finally, a fourth parametrization called "Kyber-Turbo" using the TurboSHAKE XOFs defined in [7] is considered, which employ a reduced version of the Keccak permutation with only 12 rounds, is proposed. The primitives used in each symmetric parametrization of Kyber are shown in Table 3.

Recall that hardware acceleration of Kyber can come via 2 forms: the acceleration of the symmetric primitives, as determined by each specific parametrization, or of Kyber-specific operations. Acceleration of the symmetric primitives is explored in Section IV, while the acceleration of Kyber-specific operations is explored in Section V.

## IV. ISES IN KYBER SYMMETRIC PRIMITIVES

This section illustrates the use of ISEs for hardware acceleration of symmetric primitives within each Kyber parametrization. Each subsection introduces the algorithm in question and points to where hardware acceleration takes place.

### A. KECCAK (KYBER-FIPS202 AND KYBER-TURBO)

The FIPS-202 [34] standard defines several hash and Extendable Output Functions (XOF) based on the Keccak-p[1600] (or simply Keccak) permutation and the sponge construction (chain of permutations). The width ($w$) of the Keccak permutation is determined by two parameters, rate ($r$) and capacity ($c$), such that $w = r + c$. The values of $r$ and $c$ are determined for each algorithm defined in SHA-3. The usage of these parameters for each algorithm in SHA-3 can be visualized in Table 4. For the FIPS-202 functions, the Keccak permutation has 24 iterations. Recently the TurboSHAKE functions have been proposed [7], which use the Keccak permutation with 12 iterations instead, due to the extensive security margin and existing cryptanalysis on the standard 24-round Keccak.

The Keccak permutation, operates on a 1600 bits wide state. The usage of the permutation macro $f$ in the sponge construction can be seen in Figure 3, where the input bit string is denoted by $N$, the output bit string denoted by $Z$, and the underlying permutation denoted by $f$. At each call to the permutation, $r$ bits from the input bit string are XOR-ed with the aligned output from the previous call to the permutation, until all bits from the input are fed into the sponge.

Once the input has been entirely processed, the next calls to the permutation function are used to obtain the output. Note that multiple calls to the permutation in the squeeze phase are

**TABLE 3.** Symmetric primitives for each Kyber parametrization, where XOF: eXtendable Output Function, H and G: hash functions, PRF: Pseudorandom Function, KDF: Key Derivation Function.

| Kyber Parametrization | RISC-V ISE | XOF | H | G | PRF | KDF |
|---|---|---|---|---|---|---|
| **Kyber-FIPS202** | Zbkb | SHAKE128 | SHA3-256 | SHA3-512 | SHAKE256 | SHAKE256 |
| **Kyber-90s** | Zkne + Zknh | AES-256 CTR | SHA-256 | SHA-512 | AES-256 CTR | SHA-256 |
| **Kyber-Ascon** | Xascon [15] | Ascon-XOF | Ascon-XOF | Ascon-XOF | Ascon-XOF | Ascon-XOF |
| **Kyber-Turbo** | Zbkb | TurboSHAKE128 | TurboSHAKE256 | TurboSHAKE256 | TurboSHAKE256 | TurboSHAKE256 |

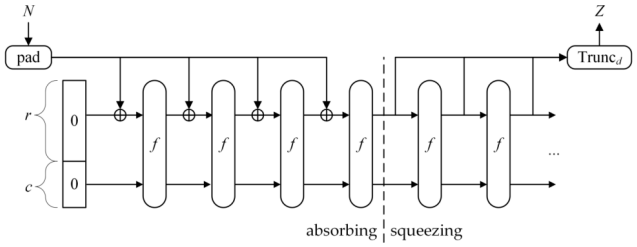



**FIGURE 3.** SHA-3 sponge construction [34].

**TABLE 4.** SHA-3 and TurboSHAKE sponge parameters.

| Hash function / XOF | $r$ | $c$ | Output Size (bits) | Security (bits) |
|---|---|---|---|---|
| SHA3-224 | 1152 | 448 | 224 | 112 |
| SHA3-256 | 1088 | 512 | 256 | 128 |
| SHA3-384 | 832 | 768 | 384 | 192 |
| SHA3-512 | 576 | 1024 | 512 | 256 |
| SHAKE128 | 1344 | 256 | - | 128 |
| SHAKE256 | 1088 | 512 | - | 256 |
| TurboSHAKE128 | 1344 | 256 | - | 128 |
| TurboSHAKE256 | 1088 | 512 | - | 256 |

possible. This is used to implement the XOFs, which can be interpreted as hash functions with arbitrarily sized output (but with finite security levels).

Internally, the Keccak permutation works on the 1600 bit state as a $5 \times 5 \times 25$ cube, shown in Figure 4. In the Keccak state cube representation, bits grouped in the $x$ dimension are rows, in the $y$ dimension are columns and in the $z$ dimension are lanes. Bits grouped in the $xy$ dimensions are slices, in the $yz$ dimensions as sheets and in the $xz$ dimensions are planes. The entire permutation consists of 5 steps, performed in the following order: $\theta$ (Theta), $\rho$ (Rho), $\pi$ (Pi), $\chi$ (Chi) and $\iota$ (Iota). For hardware acceleration with the *Zbkb* ISE, only the $\rho$ (Rho) and $\chi$ (Chi) steps are relevant.

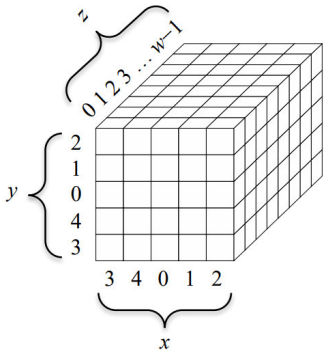



**FIGURE 4.** Keccak permutation internal state [34].

In the $\rho$ step each lane is rotated by static amounts, as shown in Figure 5. Note that these rotations are of 64 bit variables, requiring each two registers to store their value. Bit-interleaving can speed up these rotations, where instead of splitting a 64-bit variable in high and low registers, a register stores the even-indexed bits of the variable, while another stores its odd-indexed bits. A rotation by $2a$ bits in a 64-bit variable becomes two $a$ bit rotations, one in each 32-bit register [8], [20]. *Zbkb* provides the `zip`, `unzip` and `pack` instructions for the efficient translation between both representations. *Zbkb* also provides the `rori` instruction, directly implementing a rotation, otherwise implemented with two shifts and one `or` RV32I instructions.

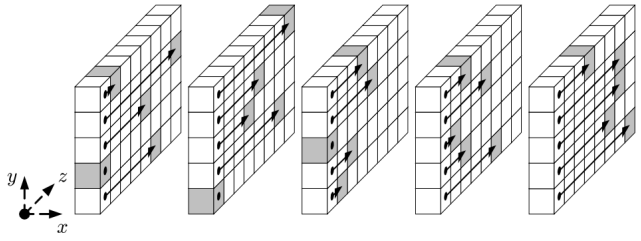



**FIGURE 5.** Keccak $\rho$ (Rho) step [34].

The $\chi$ step combines 2 lanes in the same row via bitwise AND and XOR operations. The result is placed in a lane different than the 2 input lanes, such that lanes are shuffled at the same time new values for each lane are computed, as shown in Figure 6. The *Zbkb* extension provides the `andn` instruction, which directly perform the AND bitwise operation with one negated operand.

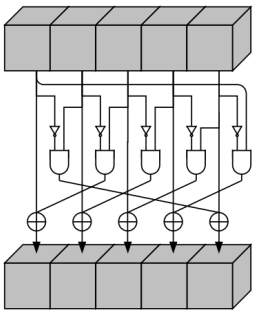


**FIGURE 6.** Keccak $\chi$ (Chi) step [34].

The SHA-3 and TurboSHAKE implementations used in this work were derived from [11], comprising both software and hardware accelerated implementations.

### B. AES AND SHA-2 (KYBER-90S)

This subsection, discussing the hardware acceleration of AES and SHA-2, has been adapted from our previous work [19].

AES [36] is a block cipher standard, widely used for confidentially transmitting information. AES iterates through its internal state in 4 fundamental operations in the following order: `SubBytes`, `ShiftRows`, `MixColumns` and `AddRoundKey`. Note that the `SubBytes` operation involves a non-trivial inversion in GF($2^8$), along with the affine transformation which follows. The entirety of the compound operation in `SubBytes` is commonly known as a Substitution Box, or SBOX. A possible technique is to pre-compute each SBOX value and store them in a small 256 bytes Look-Up Table (LUT), optimizing both code size and performance.

Seeing as the `ShiftRows` and `MixColumns` operations perform shifts and multiplications by constants, this reasoning can be taken one step further by pre-computing not the result of `SubBytes`, but of `MixColumns`. In contrast to an S-Table implementation, which used pre-computed values only for `SubBytes`, this approach is commonly known as a T-Table implementation, named following $T_0, T_1, T_2, T_3$ as defined below, where $d_{i,j}$, $a_{i,j}$ and $S$ denote the output of `MixColumns`, the round's initial state and the SBOX LUT at row $i$ and column $j$, respectively:

$$\begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} S[a_0] \\ S[a_{1,j-1}] \\ S[a_{2,j-2}] \\ S[a_{3,j-3}] \end{bmatrix} = T_0[a_0] \oplus T_1[a_{1,j-1}] \oplus T_2[a_{2,j-2}] \oplus T_3[a_{3,j-3}]$$

$$T_0[x] = \begin{bmatrix} S[x] \cdot 2 \\ S[x] \\ S[x] \\ S[x] \cdot 3 \end{bmatrix} \quad T_1[x] = \begin{bmatrix} S[x] \cdot 3 \\ S[x] \cdot 2 \\ S[x] \\ S[x] \end{bmatrix} \quad T_2[x] = \begin{bmatrix} S[x] \\ S[x] \cdot 3 \\ S[x] \cdot 2 \\ S[x] \end{bmatrix} \quad T_3[x] = \begin{bmatrix} S[x] \\ S[x] \\ S[x] \cdot 3 \\ S[x] \cdot 2 \end{bmatrix}$$

Note that each T-Table is a 1-to-4 byte mapping, instead of the previous 1-to-1 byte mapping using S-Tables. This increases memory usage from 256 to 4K bytes, but simplifies an entire AES round to 16 LUT lookups and bitwise XOR operations. The performance to code size trade-off can be balanced by pre-computing only 1 T-Table (1KB memory cost) and computing other table entries at runtime via rotations, which is the approach we use.

The *Zkne* extension defines two new specialized instructions: `aes32esmi` and `aes32esi`. `aes32esmi` can be interpreted as computing T-Table entries online in hardware, XOR-ing the current entry with previous entries for the same output column. This provides the performance benefits of a T-Table AES implementation without the need for storing LUTs in memory, leading to gains in performance and code size, as well as energy efficiency gains via the reduction in memory accesses. Finally, `aes32esi` performs only a single SBOX lookup and XOR, used in computing round keys and the last encryption round, with no `MixColumns`.

The *Zkne* instructions are implemented in the AES Unit module, shown in Figure 7. Our implementation is derived from the functional unit from the original ISE proposal in [18], which features additional logic for AES decryption and the SM4 cipher. The AES unit is entirely combinational, executing both the `aes32esmi` and `aes32esi` instructions in a single clock cycle. Both instructions are of an extended R type, which expects as inputs 2 registers to be read from the register file, plus a 2-bit immediate Byte Select (*BS*). AND gates are used to gate inputs, minimizing switching activity.

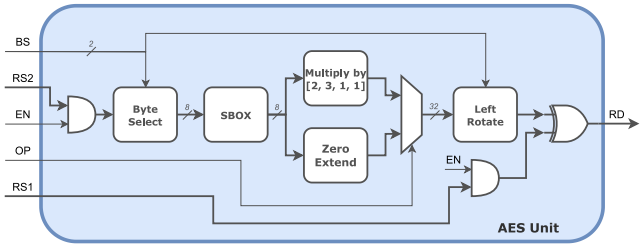



**FIGURE 7.** AES Unit [19].

Note how sub-modules in the AES Unit relate to AES fundamental operations: `SubBytes` is performed one byte at a time in the SBOX sub-module, on the byte determined by *BS* in the value read from *RS2*; `ShiftRows` is computed by selecting the appropriate byte from *RS2*, which contains a column of the current AES state, via the *BS* immediate; `MixColumns` is computed similarly to the T-Table method shown previously, where the result of `SubBytes` is multiplied by *[3, 1, 1, 2]* and rotated by *BS* bytes, computing $T_{BS}$ accounting for RISC-V little-endianess; `AddRoundKey` is computed by XOR-ing the result of `MixColumns` to the relevant word of the round key given in *RS1*. *RS1* should be equal to *RD*, such that *RD* accumulates the 4 T-Table entries and round key i.e. a column of the next round state.

The SHA-2 [35] standard defines the SHA-256 and SHA-512 hash algorithms, reducing a message of arbitrary size into 256 or 512 bits, respectively. A message M is padded and split into [256, 512] bit sized blocks $m$. An initial $h_0$ value is set, and the following blocks are computed with $h_i = f(h_{i-1}, m_i)$. The output is taken as the final $h$ value.

For SHA-256, 4 instructions are defined: `sha256sum0`, `sha256sum1`, `sha256sig0` and `sha256sig1`. These instructions directly implement the $\Sigma_0^{256}$, $\Sigma_1^{256}$, $\sigma_0^{256}$ and $\sigma_1^{256}$ functions, used in the computation of $f(h_{i-1}, m_i)$, as shown below. The $ROTR_n$ and $SHR_n$ functions denote right rotation and logical right shift by $n$ bits, respectively.

$$\Sigma_0^{256}(x) = ROTR_2(x) \oplus ROTR_{13}(x) \oplus ROTR_{22}(x)$$
$$\Sigma_1^{256}(x) = ROTR_6(x) \oplus ROTR_{11}(x) \oplus ROTR_{25}(x)$$
$$\sigma_0^{256}(x) = ROTR_7(x) \oplus ROTR_{18}(x) \oplus SHR_3(x)$$
$$\sigma_1^{256}(x) = ROTR_{17}(x) \oplus ROTR_{19}(x) \oplus SHR_{10}(x)$$

The availability of *Zknh* instructions allows for an implementation that is not only faster, but more compact and energy-efficient. Implementing these 4 functions in plain RV32I is costly due to the usage of rotations, for which no RV32I instruction can directly compute. Computing a rotation shift by $n$ bits in RV32I instructions requires a logical shift by $n$ bits in the forward direction; a logical shift by $32 - n$ bits in the reverse direction; and OR-ing both these intermediary shifts together.

Similar functions are defined for SHA-512:

$$\Sigma_0^{512}(x) = ROTR_{28}(x) \oplus ROTR_{34}(x) \oplus ROTR_{39}(x)$$
$$\Sigma_1^{512}(x) = ROTR_{14}(x) \oplus ROTR_{18}(x) \oplus ROTR_{41}(x)$$
$$\sigma_0^{512}(x) = ROTR_1(x) \oplus ROTR_8(x) \oplus SHR_7(x)$$
$$\sigma_1^{512}(x) = ROTR_{19}(x) \oplus ROTR_{61}(x) \oplus SHR_6(x))$$

Since SHA-512 variables are defined to be 64 bits in size, such functions cannot be directly implemented as a single instruction yielding a 32-bit result, as is the case for SHA-256. Six instructions are defined to accelerate SHA-512 sigma functions: `sha512sig0h`, `sha512sig0l`, `sha512sig1h`, `sha512sig1l`, `sha512sum0r` and `sha512sum1r`. With *Zknh* instructions $\sigma_0^{512}$ and $\sigma_1^{512}$ are computed with 2 different

instructions, for the high and low parts of the result separately, while $\Sigma_0^{512}$ and $\Sigma_1^{512}$ are computed via 2 executions of the same instruction, but once with inverted operands, implying a rotation by 32 bits.

*Zknh* instructions are also implemented in their own functional unit, shown in Figure 8. The SHA-2 unit is entirely combinational, with all *Zknh* instructions executing in a single cycle. The SHA-2 sigma functions lend themselves to an efficient hardware implementation, seeing as shifts and rotations by static amounts have zero cost. The SHA-2 unit does not have a well-defined internal structure like the AES unit, and can be seen simply as a sea of XOR gates.

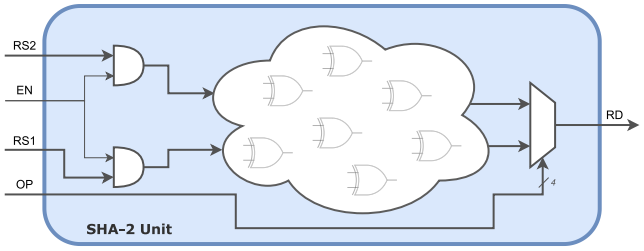



**FIGURE 8.** SHA-2 Unit [19].

The AES software implementation used in this work uses the balanced T-Table technique, which stored in memory only a single 1KB LUT. It was derived from [10], which uses the full 4KB T-Tables. The SHA-2 software implementations were adapted from the Tinycrypt library [37]. Hardware accelerated AES is implemented directly in assembly while hardware accelerated SHA-2 is implemented with inline assembly statements in the base software implementation.

### C. ASCON (KYBER-ASCON)

This subsection, which discusses the hardware acceleration of Ascon, has been adapted from our previous work [20].

Like SHA-3, Ascon [4] is also based on the sponge construction, using a different permutation as well as different rate ($r$) and capacity ($c$) values. The Ascon sponge is depicted in Figure 9. At each stage of the sponge, a permutation of the internal state is performed ($p^a$ and $p^b$, which differ only in the number of rounds). Internally, the permutation has a state of 320 bits, split into five 64-bit variables $x_0, \ldots, x_4$. Figure 10 summarizes the Ascon permutation operation. This work considers the Ascon-Hash and Ascon-XOF variants, with the sponge rate ($M_i$, and $H_i$ width) of 64 bits.

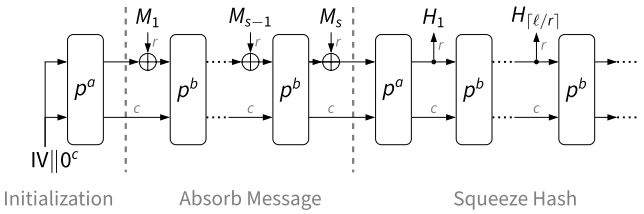
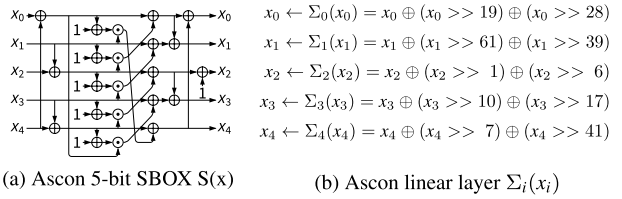



**FIGURE 9.** Ascon sponge [4].

Hardware acceleration of the Ascon permutation is done mainly in the linear diffusion (Figure 10(b)) step of the Ascon




(a) Ascon 5-bit SBOX S(x) (b) Ascon linear layer $\Sigma_i(x_i)$

**FIGURE 10.** Ascon substitution layer and linear diffusion layer [4].

permutation. The linear diffusion step consists of rotations by constant amounts and bitwise XORs of these rotations in sigma functions, like those in the SHA-2 hash functions.

With the *Xascon* extension, sigma functions can be computed directly with two instructions in the same manner as the SHA-512 instructions from the standardized *Zknh* extension. The hardware implementation of *Xascon* instructions follows that of *Zknh* instructions, being simply a sea of XOR gates as depicted in Figure 8.

The Ascon software implementation used in this work was derived from [10], removing loop unrolling. The hardware accelerated implementation is obtained via inline assembly statements for the linear diffusion layer in the Ascon permutation in the base software implementation.

## V. XKYBER ISE FOR KYBER-SPECIFIC OPERATIONS

When proposing new RISC-V extensions, several principles should be followed to design new scalar instructions that follow the overall design philosophy of the RISC-V architecture. These principles, concisely stated by [17] in the context of the original proposals for an AES ISE are restated below. Note that previous proposals reviewed in Section II do not adhere to these principles, such as [26], [27], and [31].

- The ISE must align with the wider RISC-V design principles. This means it should favour simple building-block operations, and use instruction encodings with at most 2 source registers and 1 destination register. This avoids the cost of a general-purpose register file with more than 2 read ports or 1 write port.
- The ISE must use the RISC-V general-purpose scalar register file to store operands and results, rather than any vector register file (or floating-point register file).
- The ISE must not introduce a special-purpose architectural state, nor rely on a special-purpose micro-architectural state (e.g., caches or scratch-pad memory).
- The ISE must enable data-oblivious execution (in [17], of AES), preventing timing attacks based on execution latency (e.g., from accesses to a pre-computed S-box).
- The ISE must be efficient, in terms of improvement in execution latency per area required: this balances the value in both metrics vs. an exclusive preference for one or the other. Efficiency wrt. auxiliary metrics, e.g., memory footprint or instruction encoding points, is an advantage but not a requirement.

In this work the *Xkyber* extension is proposed, containing 6 new instructions designed to optimize Kyber implementations in low cost RISC-V 32 bit processors, following the

general principles outlined above. Four of these instructions rely on the fact that Kyber polynomial coefficients are 12 bit integers, stored in memory as arrays of 16 bit variables, meaning that 2 coefficients can fit in the native 32 bit word size. Processing can be performed in parallel for 2 coefficients at a time using regular integer arithmetic elements in the base processor, without the need for costly vector co-processors or extensive additional logic. Single Instruction Multiple Data (SIMD) processing also reduces the amount of memory accesses, seeing as 2 coefficients are loaded/stored from/to memory in a single word-wide access. The 6 new instructions are presented in Table 5.

The *Xkyber* implementation and integration into Ibex is illustrated in Figure 11, along with other previously discussed ISEs, shown with dotted lines. Existing elements are shown in green, added elements in blue and control signals in orange. ISEs *Zknh* (SHA-2 sigma functions), *Xascon* (Ascon sigma functions), *Zkne* (AES) are implemented as functional units, while *Zbkb* (bit-manipulation) is implemented in the ALU.

As Figure 11 shows, we employ fine-grained, tightly coupled acceleration. Fine-grained acceleration executes only the most critical parts of the algorithms in dedicated hardware. Tightly coupled accelerators are closely integrated into the processor, sharing resources with its main architectural blocks. This contrasts with the co-processor acceleration approach, which implements the acceleration in a dedicated hardware block. Our approach reduces memory accesses and processing power, particularly advantageous for resource-constrained embedded environments.

The adoption of fine-grained, tightly coupled acceleration effectively increases resource sharing. For the *kyberadd* and *kybersub* instructions, the integer ALU is used to compute the pre-mod-q intermediary result of the addition/subtraction. For the *kybermul* and *kybercompress* instructions, the 16-bit multiplier is reused. All *Xkyber* instructions share the conditional subtraction (mod q) adder and mux, *kybermul* and *kybercompress* share the 5039 constant multiplier.

The `kybercbd2` and `kybercbd3` instructions implement CBD sampling as shown in Algorithm 1, sampling 2 coefficients $\in \mathbb{Z}_q$ at a time. The coefficients are placed in the upper and lower 16 bits of the target register, facilitating storing both coefficients in memory with a single word-wide store instruction. For $CBD_3$, 2 full adders are needed to

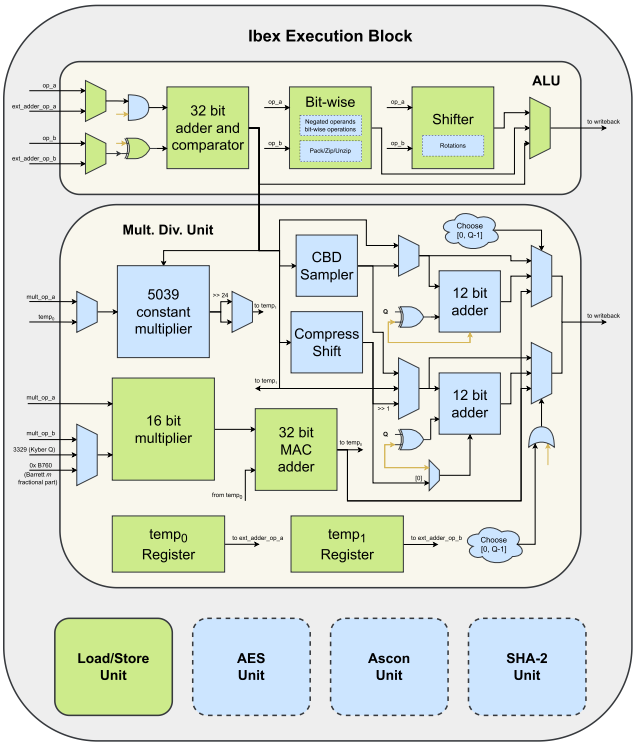



**FIGURE 11.** Ibex execution block with extensions.

compute the $A$ and $B$ in Algorithm 1, as well as a 3-bit adder and 3 inverters to compute the $A - B$ difference for each of the 2 $CBD_3$ computations. For $CBD_2$, 2 half adders are enough to compute $A$ and $B$. Once $A - B$ is computed, the result in $\{-\eta, \ldots, \eta\}$ must be brought to the $\{0, \ldots, \eta, q - \eta - 1, \ldots, q-1\}$ range by conditionally adding $q$ if $A-B < 0$. The conditional addition logic is shared with the `kyberadd`, `kybersub` and `kybermul` instructions.

`kyberadd` and `kybersub` compute 2 coefficient additions or subtractions in $\mathbb{Z}_q$ in parallel. In this manner two coefficients can be fetched and stored back in memory in single word-wide instructions in addition to being processed simultaneously. The additions/subtractions themselves are performed in the existing 32 bit adder in the ALU, with trivial masking for stopping carry propagation from the coefficient in the lower bits to the coefficient in the higher bits. Like with `kybercbd2` and `kybercbd3`, *mod q* is performed

**TABLE 5.** Xkyber instructions.

| Instruction | Functionality |
|---|---|
| kybercbd2 RD, RS1 | $RD[11{:}0] \leftarrow CBD_2(RS1[3{:}0]); RD[27{:}16] \leftarrow CBD_2(RS1[7{:}4]);$ (Algorithm 1) |
| kybercbd3 RD, RS1 | $RD[11{:}0] \leftarrow CBD_3(RS1[5{:}0]); RD[27{:}16] \leftarrow CBD_3(RS1[11{:}6]);$ (Algorithm 1) |
| kyberadd RD, RS1, RS2 | $RD[11{:}0] \leftarrow RS1[11{:}0] + RS2[11{:}0]\ mod\ q; RD[27{:}16] \leftarrow RS1[27{:}16] + RS2[27{:}16]\ mod\ q;$ |
| kybersub RD, RS1, RS2 | $RD[11{:}0] \leftarrow RS1[11{:}0] - RS2[11{:}0]\ mod\ q; RD[27{:}16] \leftarrow RS1[27{:}16] - RS2[27{:}16]\ mod\ q;$ |
| kybermul RD, RS1, RS2 | $RD \leftarrow RS1[11{:}0] \times RS2[11{:}0]\ mod\ q;$ |
| kybercompress RD, RS1, RS2 | $RD \leftarrow Compress_{RS2}(RS1[11{:}0]);$ (Algorithm 2) |

via conditional addition or subtraction, subtracting $q$ if if the intermediary result is $> q$ in the `kyberadd` case or adding $q$ if the intermediary result is $< 0$ in the `kybersub` case.

`kybermul` performs a single (non-SIMD) multiplication in $\mathbb{Z}_q$. Note than the modular reduction cannot be performed as a simple conditional subtraction as previously done with `kyberadd` due to the fact that the intermediary multiplication is in the $\{0, \ldots, (q-1)^2\}$ range, where a conditional subtraction by $q$ could only bring it to the $\{0, \ldots, (q-1)^2 - q\}$ range. For efficiently computing the modular reduction of the intermediary product, the Barrett reduction [38] is used. The generic algorithm for the Barrett reduction is presented in Algorithm 3. In this work the $k$ variable is set as 24.

**Algorithm 3** Multiplication Mod $q$ (Barrett Reduction [38])

**Require:** $a, b \in \mathbb{Z}_q, k \approx 2 \cdot \lceil log_2\, q \rceil, m = \lfloor 2^k/q \rfloor = 5039$
**Ensure:** $z \in \mathbb{Z}_q$
$temp_0 \leftarrow a \cdot b$
$temp_1 \leftarrow (temp_0 \cdot m) \gg k$
$temp_1 \leftarrow (temp_1 \cdot q)$
**return** $((temp_0 - temp1) \geq q)\ ?\ (temp_0 - temp_1) - q\ :$
$(temp_0 - temp_1)$

The modular multiplication with Barrett reduction is implemented in 4 clock cycles, making extensive use of resource sharing with existing elements in the Ibex processor's 3-cycle multiplier and 2 internal temporary registers $temp_0$ and $temp_1$ for intermediary results. Resource-sharing with added elements is also explored, namely in the lower conditional subtraction adder/mux also used in all other *Xkyber* instructions. The computations performed at each clock cycle follow the structure presented in Algorithm 3, with one line of Algorithm 3 being computed per clock cycle.

First, the 12 bit by 12 bit multiplication in line 1 is computed in the 16 bit multiplier, with its result being stored in $temp_0$, exactly like the first cycle of 32 bit by 32 bit multiplications from the *M* extension.

In the second cycle, the product of $temp_0$ and the $m$ constant is computed via a constant multiplier. The multiplier implementation stems from the fact that the 5039 constant can be factored as sums and subtractions of powers of two in the form $5039 = 2^{12} + 2^{10} - 2^6 - 2^4 - 2^0$. Seeing as multiplications by powers of two can implemented as trivial left-shifts, a multiplication by 5039 can then be written as $x \cdot 5039 = (x \ll 12) + (x \ll 10) - (x \ll 6) - (x \ll 4) - x$. In this work, the $(x \ll 12) + (x \ll 10)$ share is computed in the existing Ibex ALU, requiring a single additional 32 bit multiplexer such that one of the operands is $(x \ll 2)$, while the other operand is simply $x$. This result, left-shifted by 10, is added to $(x \ll 6) - (x \ll 4) - x$ as computed in 3 new adders instantiated in the multiplier. In this manner, the additional resources required for the 5039 constant multiplier are only 3 adders and a multiplexer. At the end of the second cycle, $temp_1$ is written the result of the constant 5039 multiplication shifted by $k = 24$.

In the third cycle of `kybermul` the multiplication of $temp_1$ and the $q$ constant is performed. This multiplication is computed in the 16 bit generic multiplier, requiring the addition of additional multiplexers to its existing mux logic, but not a second dedicated constant multiplier. Note that the $q$ constant mux is a logical mux, not a physical mux, since it needs to multiplex a constant value. This result is written to the $temp_1$ register.

The fourth and final cycle of `kybermul` computes the subtraction of $temp_1$ from $temp_0$ and fits its range into $\{0, \ldots, q-1\}$. $temp_0 - temp_1$ is computed in the ALU, while the range adjustment is computed using the same logic as other *Xkyber* instructions.

Finally, `kybercompress` performs the (lossy) compression of a 12 bit coefficient into a $d$ bit value. Like `kybermul`, it is also a non-SIMD instruction. First, note that the Barrett reduction constant $m = \lfloor 2^k/q \rfloor$, as shown in Algorithm 3, is similar to the $2^d/q$ multiplication in Algorithm 2. Also note that compression requires rounding to the nearest integer, while $m$ is truncated. In order to reuse the $m = 5039$ constant multiplier for computing coefficient compression, the $2^d/q$ multiplication is computed using both the constant multiplier and the generic 16 bit multiplier. In this manner, seeing as the $2^d/q$ multiplication can be rewritten as $2^k/q \gg (k-d)$, the constant 5039 multiplier computes multiplication for the integer part while the generic multiplier computes multiplication by the fractional part, storing the results in $temp_0$ and $temp_1$, respectively.

In the second and final clock cycle of `kybercompress`, both integer and fractional shares of the $2^d/q$ multiplication are added, and shifted by $24-d-1$. The LSB of this intermediary shift value determines if $+1$ will be added to the final result, computing the nearest integer rounding. The $+1$ addition to the combined integer and fractional shares shifted by $24-d$ is computed in the range adjusting adder, using only the carry in bit with the $q$ constant masked by AND gates. Finally, the adjust adder result is masked by $d$ bits, effectively computing $mod\ 2^d$, yielding the final result.

## VI. EXPERIMENTAL EVALUATION

In this section, a comparative evaluation of the Kyber-512, Kyber-768, and Kyber-1024 security levels is presented, considering the symmetric parametrizations Kyber-FIPS202, Kyber-90s, and the novel Kyber-Ascon and Kyber-Turbo for each of the `KeyGen`, `Enc`, `Dec` steps. The evaluation is performed with and without hardware acceleration for the symmetric primitives and Kyber-specific operations via the *Xkyber* extension. The evaluation comprises a performance comparison as well as energy consumption, memory footprint and die area cost analyses.

### A. EVALUATION METHODOLOGY

Our evaluation is performed considering the round 3 Kyber reference implementation as submitted to the NIST PQC

competition. The symmetric primitives in Kyber are provided by the Tinycrypt library, using a modified AES implementation with a 1KB T-Table (Kyber-90s), the SHA-3 implementation from Saarinen [11] and a custom compact Ascon implementation based on Campos et al. [10]. Random number generation is accomplished via a NIST SP-800-90A [39] compliant AES-256 in CTR mode based PRNG (CTR_DRBG) from Tinycrypt. All of these implementations, including the Kyber round 3 reference implementation, are modified to support hardware acceleration via RISC-V ISEs. Resulting Kyber implementations are validated with test vectors generated by the reference implementation submitted to the third round of the NIST competition compiled and executed outside of our simulation environment.

For Kyber-FIPS202 or Kyber-90s, no deep changes to the reference implementation are necessary. For Kyber-Ascon the reference C implementation[1] was integrated into the same Kyber implementation, while for Kyber-Turbo trivial modifications were made as to reduce the number of rounds of the Keccak permutation to 12 and replace SHA-3 hash functions with TurboSHAKE256 [7]. In all cases the symmetric primitive implementation used for generating test vectors is different from those in the simulation environment, ensuring correctness.

In the simulations from where we obtain our measurements, Kyber is enforced to withstand remotely exploitable timing side-channel attacks by showing a constant-time execution by running simulations with the Ibex data independent timing CSR being set to 1, as expected in a real-world deployment. Protection against higher effort attacks that require physical access to the device such as power analysis or fault injection attacks are outside of the scope of this work. Power analysis attacks could be mitigated by using the Ibex dummy instruction insertion feature.[2]

The evaluation setup for all experiments performed in this work uses the Ibex Simple System,[3] a minimal implementation of a practical system around the Ibex processor, including the essential elements necessary for running software on Ibex. Besides the Ibex processor itself, it contains a 2-port RAM, connected to the Ibex instruction and data memory interfaces; a programmable timer; and a control peripheral that can stop the simulation and write ASCII output from the processor into a file in the host machine for the simulation session.

The software build process employs GCC version 12.2.0 with the *-Os* flag, generating binaries optimized for code size. For the non-standard *Xascon* and *Xkyber* extensions, assembly using pre-built toolchains is impossible, seeing as the toolchain does not have knowledge of the instructions in these extensions. A custom *binutils* build supporting *Xascon* and *Xkyber* opcodes is necessary.

The Ibex processor contains a set of performance counters[4] that can provide insights into the run-time behavior of software being executed. In addition to the standard Ibex counters, 6 new counters were added, such that loads and stores at word, half-word, and byte granularity can be differentiated. A profiler tool was implemented to accurately characterize the software being executed, which can capture the state of the performance counters during the simulation without software intervention. The profiler works at C function granularity, computing the difference between counter values at function entry and exit for a given list of functions of interest. In addition to performance counter outputs, the tool provides the simulation time for each entry and exit event.

Function entry and exit events are determined by monitoring the Program Counter (PC) register, which contains the address of the instruction being executed in the processor. A function entry event is seen when the PC register takes in the starting address of one of the C functions of interest. Each starting address is known via the ELF executable generated by the build process using the GNU *nm* tool. This process is entirely automated in the build environment; the user must only provide a list of function names, for which a file containing the starting address for each function is generated and read by the profiler at the beginning of the RTL simulation. Conversely, function exit is determined by checking the PC register for the address immediately after the `JAL` instruction that leads to a function entry event.

Memory footprint is evaluated considering code size, static data, and maximum stack usage. Code size and static data are known through section sizes obtained via a custom script that parses the sections header of disassembled object code generated by the *objdump* tool, with *-fhSD* flags. Code is compiled with the *-fdata-sections* and *-ffunction-sections* GCC flags, creating individual sections for each function and data symbol instead of lumping them together in a single *.text* or *.data* section. As the reference Kyber implementation is not memory-optimized, we use stack data from [40], which provides a better estimate for real-world embedded implementations.

Hardware-oriented metrics are evaluated using netlists generated by synthesis with Cadence Genus version 2112. A high-density 8-track cell library in 28 nm technology from STMicroelectronics is used. Synthesis considers a PVT corner of slow process, 0.75 V at 125 C. We use the worst-case RC product QRC techfile for estimating interconnect delays. Netlist simulations are performed using the aforementioned simulation infrastructure. The average power of the Ibex core during a specific function call can be estimated using the collected switching activity of the

[1] https://github.com/ascon/ascon-c/tree/main/crypto_hash/asconhashv12/ref

[2] https://ibex-core.readthedocs.io/en/latest/03_reference/security.html#dummy-instruction-insertion

[3] https://github.com/lowRISC/ibex/tree/master/examples/simple_system

[4] https://ibex-core.readthedocs.io/en/latest/03_reference/performance_counters.html

synthesized netlist, considering the time window determined by the profiler tool. We obtain the energy consumed by the core during the function call from the average power during the time window known from the profiler tool, multiplied by the time window size. Power analysis with Cadence Joules considers a PVT corner of nominal process, 0.9 V at 25 C and nominal QRC techfile.

To obtain a more comprehensive energy cost evaluation, memory accesses are considered in addition to the processor core costs. The CACTI 7 [41] tool is used to estimate the energy spent by memory reads and writes. The tool is configured to provide estimates for a 16 KB memory with a read-only port and a second read/write port, considering low-power bitcells at the 28 nm technology node. We obtain energy cost values of 63.362 fJ/bit for reads and 41.436 fJ/bit for writes. The number of read and write to the memory is determined via the performance counters output of the profiler tool. The relevant counters are the total amount of instructions and the number of compressed instructions fetched during the function call, as well as amounts of loads and stores, differentiated by word, half-word, and byte granularity.

### B. PERFORMANCE EVALUATION AND PROFILING

Kyber performance relies on symmetric primitives for its internal sampling and hashing operations, shown in Table 3. Table 6 shows a throughput comparison between symmetric primitives (XOF, PRF, H, G, KDF) choices used within Kyber. White and blue columns show values for software-only and hardware accelerated implementations, respectively.

From Table 6, we can see:

- AES-256 in CTR mode (Kyber-90s) shows the best throughput out of all primitives for sampling the **A** matrix (XOF primitive), considering both software-only and hardware-accelerated implementations;
- For the G and H hash functions, SHA-256 and SHA-512 (Kyber-90s) are faster than SHA3-256 and SHA3-512 (Kyber-FIPS202) and TurboSHAKE256 (Kyber-Turbo), for both software-only and hardware-accelerated implementations, again favoring Kyber-90s over other parametrizations;
- Software-only Ascon-Hash and Ascon-XOF (Kyber-Ascon) have better performance than SHA3-512 but worse than SHA3-256 (Kyber-FIPS202). However, hardware accelerated Ascon-Hash and Ascon-XOF outperform both SHA3-256 and SHA3-512.

Figure 12 shows the performance of each of the `KeyGen`, `Enc`, `Dec` steps (horizontally) in the Kyber-[512, 768, 1024] security levels (vertically), for each parametrization.

- Kyber-90s is the fastest Kyber parametrization, considering both software-only and hardware accelerated implementations, with software-only Kyber-90s being faster than hardware accelerated Kyber-FIPS202 and Kyber-Ascon (Kyber-90s `Enc` is 30% (software) 42% (ISE) faster than Kyber-FIPS202 `Enc` for Kyber-512);
- Kyber-Turbo is significantly faster than Kyber-FIPS202 (22% (software) 24% (ISE), Kyber-512 `Enc`);
- Kyber-Ascon has comparable performance to Kyber-FIPS202 considering software-only implementations, but is significantly faster when comparing hardware accelerated implementations (17% Kyber-512 `Enc`);.
- Kyber-Ascon is the parametrization that benefits the most from ISEs, showing an improvement of over 29% in clock cycle count in Kyber-512 encapsulation;
- Hardware acceleration of AES in Kyber-90s leads to bigger gains than of SHA-2. For Kyber-512 `Enc`, with *Zkne* a 23% gain in clock cycle count is observed, while *Zkne* + *Zknh* shows only an additional 7% gain compared to *Zkne* only.

Figure 13 shows the performance of the 3 Kyber steps using the *Xkyber* extension. A baseline software implementation of Kyber-90s is compared to an implementation with only the symmetric primitives accelerated via the *Zkne* and *Zknh* extensions and an implementation with both symmetric acceleration and Kyber-specific hardware acceleration via *Xkyber*. The impact of accelerating Kyber operations via *Xkyber* is roughly the same as accelerating symmetric primitives via *Zkne* and *Zknh*. From Figure 12, Kyber-90s is the best case for hardware acceleration of symmetric primitives. If the Kyber variant were Kyber-FIPS202, Kyber-Turbo or Kyber-Ascon, hardware acceleration via *Xkyber* would outperform acceleration of symmetric primitives via *Zbkb* or *Xascon*.

Figure 14 shows the performance of Kyber operations accelerated by the *Xkyber* extension. Significant gains are are shown in each of the operations, with the most significant clock cycle gains being in polynomial pointwise multiplication, (67% gain), CBD3 (72% gain) and polynomial coefficient compression (Algorithm 2) $\text{Compress}_4$ (78% gain).

### C. MEMORY FOOTPRINT EVALUATION

Considering only the Kyber implementation by itself, memory footprint is not significantly influenced by the choice of symmetric primitives or the hardware acceleration via ISEs

**TABLE 6.** Core operation profiling for each Kyber XOF, PRF, H, G, KDF choice (smaller is better). Hardware accelerated implementations in blue columns.

| Metric | AES-256 CTR | | SHA-256 | | SHA-512 | | SHA3-256/SHAKE256 | | SHA3-512 | | SHAKE128 | | TurboSHAKE128 | | TurboSHAKE256 | | Ascon-Hash/Ascon-XOF | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Cycles | 2,056 | 425 | 6,237 | 4,181 | 17,217 | 11,087 | 30,197 | 22,355 | 30,197 | 22,355 | 30,197 | 22,355 | 18,588 | 11,767 | 18,588 | 11,767 | 1,610 | 799 |
| Instr. | 1,722 | 326 | 5,621 | 3,569 | 15,243 | 9,623 | 22,715 | 13,713 | 22,715 | 13,713 | 22,715 | 13,713 | 14,432 | 7,278 | 14,432 | 7,278 | 1,527 | 695 |
| Cycles/byte | 128.5 | 26.6 | 97.5 | 65.3 | 134.5 | 86.6 | 222.0 | 164.4 | 419.4 | 310.5 | 179.7 | 133.1 | 110.6 | 70.0 | 136.7 | 86.5 | 201.2 | 99.9 |
| Instr./byte | 107.6 | 20.4 | 87.8 | 55.8 | 119.1 | 75.2 | 167.0 | 100.8 | 315.5 | 190.5 | 135.2 | 81.6 | 85.9 | 43.3 | 106.1 | 53.5 | 190.9 | 86.9 |

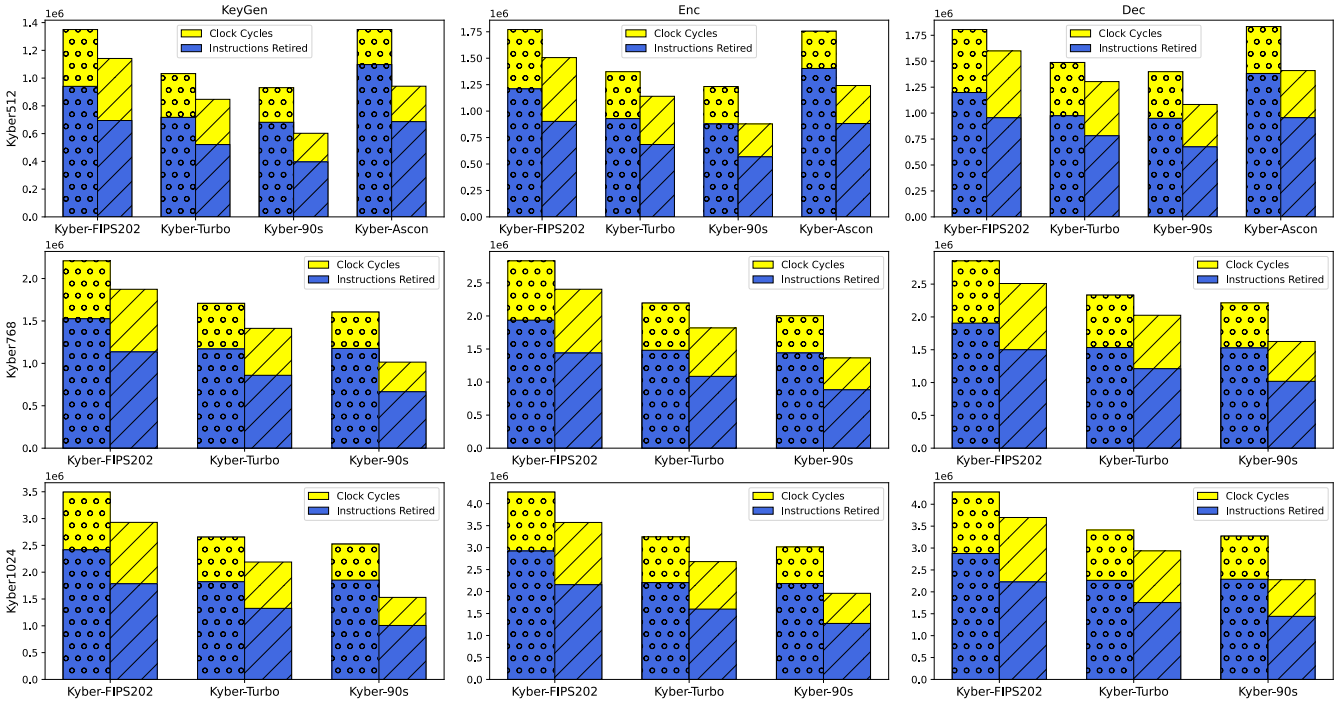



**FIGURE 12.** Performance of hardware acceleration of Kyber symmetric primitives (left columns = software implementations, right = hardware accelerated).

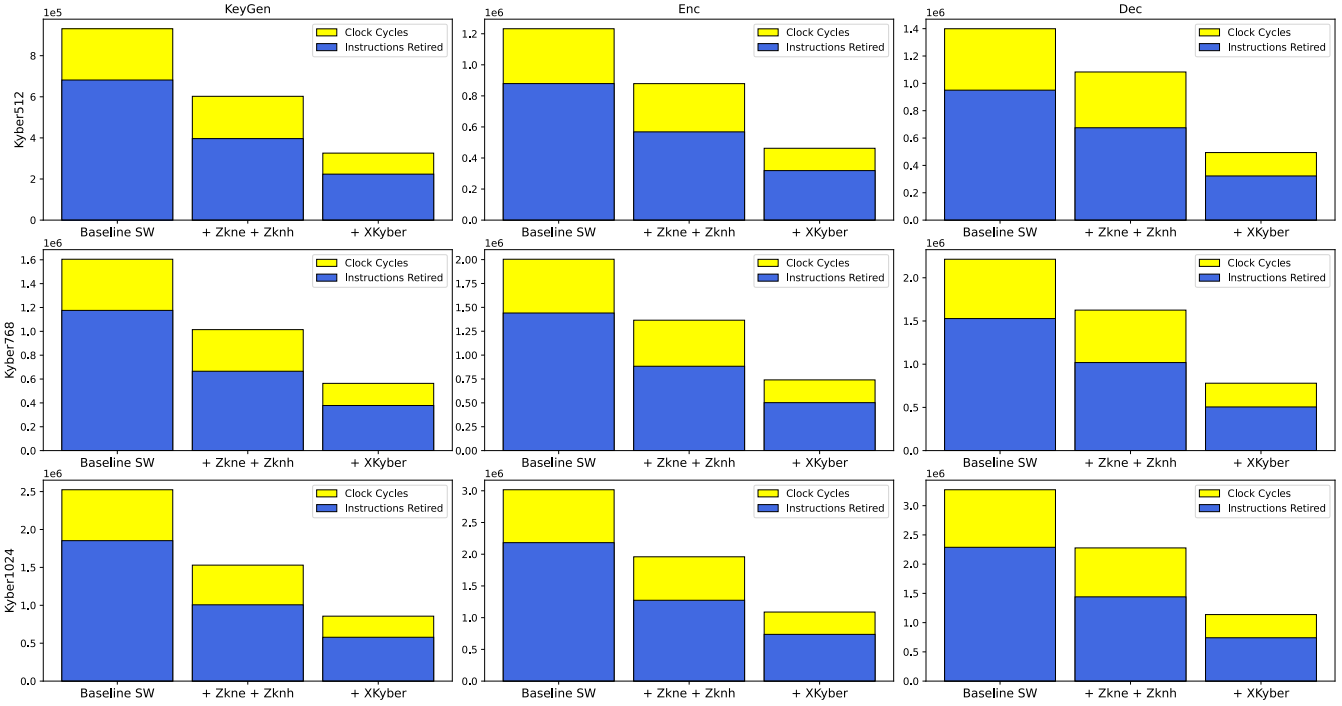



**FIGURE 13.** Performance with Xkyber extension in Kyber-90s (note the scale difference in each plot) for each of the KeyGen, Enc, Dec steps (horizontally) in the Kyber-[512, 768, 1024] security levels (vertically).

of those primitives. However, the internal Kyber operations have reduced code size as shown in Figure 15. The baseline software implementations are shown with bars hatched with circles, while hardware accelerated implementations using instructions from the *Xkyber* extension are shown in bars hatched with diagonal stripes. Most operations have a noticeable code size decrease, most significantly INTT and CBD3.

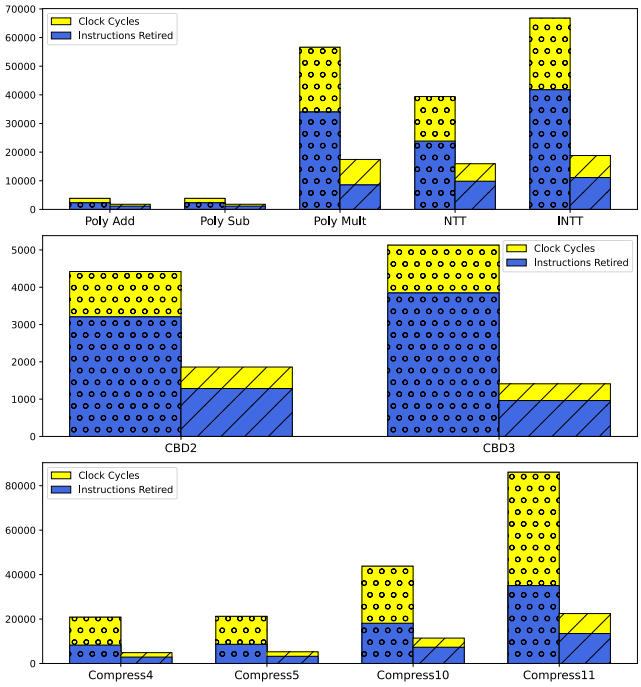



**FIGURE 14.** Performance with Xkyber extension in Kyber internal ops.

Additionally, auxiliary functions such as Barrett and Montgomery reductions and conditional subtractions are not needed anymore, seeing as operands with *Xkyber* instructions are always $\in \mathbb{Z}_q^n$. A software-only implementation of Kyber-512 with the Kyber-FIPS202 parametrization shows a total code size of 5,090 bytes (not including symmetric primitives), while an implementation with the *Xkyber* extension shows a code size of 4,314 bytes, a 15% gain.

A more comprehensive memory footprint analysis can be performed considering the entire cryptosystem, not only the Kyber implementation but the symmetric primitives used in Kyber as well. Table 7 compares 4 different practical cryptosystems considering authenticated encryption (AES in CCM mode or Ascon-128), hash functions (SHA-2, SHA-3, TurboSHAKE or Ascon-Hash) and KEM (Kyber-512).

- Comparing software-only implementations, the Kyber-90s cryptosystem shows the biggest memory footprint (17,226 bytes), due to the AES T-Table and SHA-2 round constants, as well as no code sharing between SHA-256 and SHA-512 like SHA3-256 and SHA3-512 are able to, sharing the Keccak permutation and the sponge structure;
- Hardware acceleration brings small gains in memory footprint for the cryptosystems based on Kyber-FIPS202 and Kyber-Turbo (1%), but bigger gains (14%) for the Kyber-90s cryptosystem, such that they show a similar memory footprint;
- Conversely, the cryptosystem based on Kyber-Ascon is the most compact (13,472 SW / 13,234 ISE). Our Ascon implementation could be further optimized for code size, sharing sponge absorb/squeeze logic between authenticated encryption/decryption and hashing/XOF.

### D. DIE AREA COSTS EVALUATION

Table 8 shows synthesis results for the Ibex core and each ISE combination being explored. For the *Zkne* and *Zknh* combination a 15% increase is observed (Kyber-90s), while *Xascon* shows a 12% increase (Kyber-Ascon). *Zbkb* shows a smaller 5% cell area increase (Kyber-FIPS202 and Kyber-Turbo). *Xkyber* shows a similar increase of 4.3K equivalent gates as the *Zkne* and *Zknh* case, while also providing similar performance improvements.

In terms of performance gains per die area cost, the most cost-effective manner of accelerating Kyber via ISEs is accelerating AES via the *Zkne* extension. The use of *Zkne* AES in the Kyber-90s parametrization brings 27% gains in Kyber-512 `Enc` performance at the cost of 4% additional die area. The least efficient form of hardware acceleration comes from using the *Zknh* extension for accelerating the SHA-2 hash functions in Kyber-90s, bringing 7% gains in Kyber-512 `Enc` performance at the cost of 10% additional area, when compared to the *Zkne*-only case.

The parametrization that benefits the most from hardware acceleration in terms of energy consumption gains per area cost is also Kyber-90s, showing a 25% energy consumption improvement for Kyber-512 encapsulation at the cost of 4% additional area.

**TABLE 7.** Kyber-512 cryptosystem total memory footprint in bytes (white = software, blue = [Zbkb, Zkne + Zknh, Xascon], green = Zkne + Zknh + Xkyber).

| Cryptosystem | AES CCM + SHA-3 + Kyber-FIPS202 | | AES CCM + TurboSHAKE + Kyber-Turbo | | AES CCM + SHA-2 + Kyber-90s | | | Ascon-128 + Ascon-Hash + Kyber-Ascon | |
|---|---|---|---|---|---|---|---|---|---|
| **Stack usage** [40] | 3,136 | 3,136 | 3,136 | 3,136 | 3,136 | 3,136 | 3,136 | 3,136 | 3,136 |
| **Code size** | 10,238 | 10,076 | 10,238 | 10,076 | 11,842 | 10,490 | 9,788 | 10,080 | 9,842 |
| **Static data** | 1,480 | 1,480 | 1,384 | 1,384 | 2,248 | 1,223 | 1,223 | 256 | 256 |
| **Total** | 14,854 | 14,692 | 14,758 | 14,596 | 17,226 | 14,849 | 14,147 | 13,472 | 13,234 |

**TABLE 8.** Area comparison of the Ibex processor with ISEs for Kyber hardware acceleration.

| Synthesis results | Ibex | Ibex Zbkb | Ibex Zknh + Zkne | Ibex Zkne | Ibex Xascon | Ibex Zknh + Zkne + Xkyber |
|---|---|---|---|---|---|---|
| **Cell Area** ($\mu m^2$) | 10810 | 11368 | 12481 | 11212 | 12118 | 13873 |
| **Net Area** ($\mu m^2$) | 5754 | 5572 | 6883 | 5231 | 8371 | 8038 |
| **Total Area** ($\mu m^2$) | 16564 | 16940 | 19364 | 16443 | 20490 | 21911 |
| **Cell Count** (# instances) | 10530 | 11673 | 11830 | 11310 | 10994 | 12799 |
| **Equivalent NAND2 gates** | 33120 | 34828 | 38239 | 34351 | 37127 | 42503 |
| **Slack @ 500 MHz** (ns) | 0 | 0 | 0 | 0 | 0 | 0 |

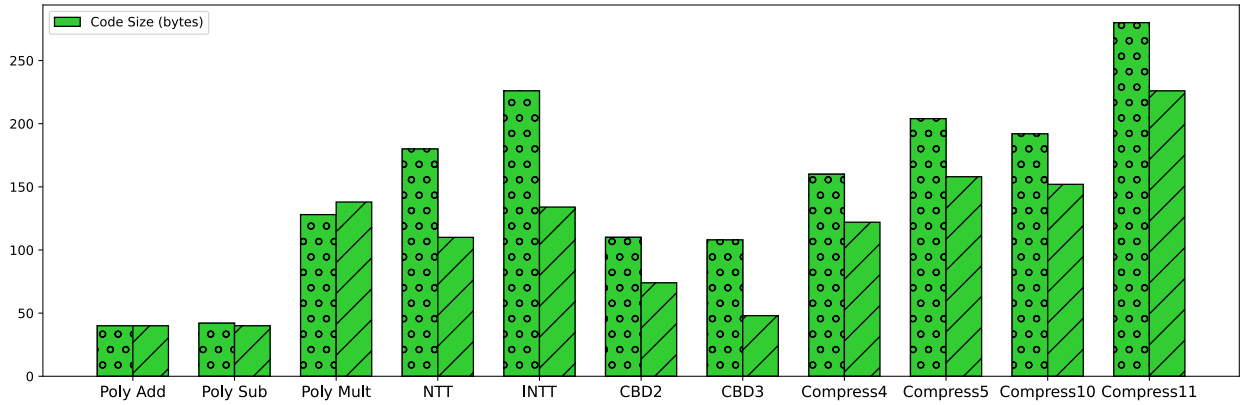



**FIGURE 15.** Code size of Kyber operations accelerated by Xkyber (left columns = software implementations, right = hardware accelerated).

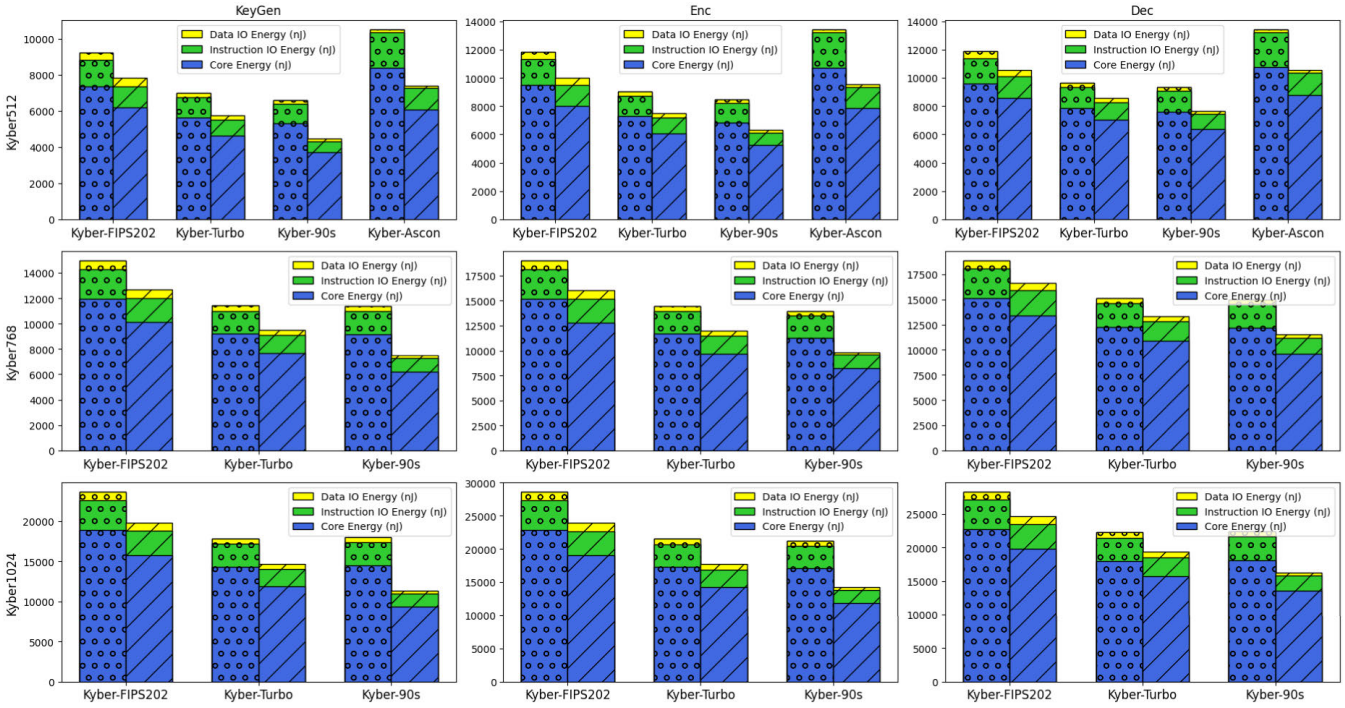



**FIGURE 16.** Energy consumption with hardware acceleration of Kyber symmetric primitives for each of the KeyGen, Enc, Dec steps (horizontally) in the Kyber-[512, 768, 1024] security levels (vertically). Left columns = software implementations, right = hardware accelerated.

### *E. ENERGY CONSUMPTION EVALUATION*

The energy consumption of each Kyber step in each Kyber symmetric parametrization is shown in Figure 16. Energy spent due to data and instructions memory operations are shown, respectively, in yellow and green. Energy spent due to data processing inside the Ibex processor is shown in blue. The baseline software implementations are hatched with circles, while implementations with hardware-accelerated symmetric primitives are hatched with diagonal stripes.

Core energy dominates the total energy consumption due to data processing rather than memory operations. Despite having roughly the same performance, Kyber-Ascon shows higher energy consumption than Kyber-FIPS202, comparing software implementations. Kyber-Ascon executes more instructions per clock cycle, increasing average power, and therefore, total energy. Kyber-Ascon shows the most benefits from ISEs, showing gains of 29% in energy consumed in Kyber-512 encapsulation. Software Kyber-90s shows slightly better energy consumption than hardware-accelerated Kyber-FIPS202, further improving considering hardware acceleration. For Kyber-512 encapsulation, Kyber-90s consumes 28% (software) and 37% (ISE) less energy.

Figure 17 shows the energy consumption of each Kyber step and security level, considering a software-only Kyber-90s baseline, hardware accelerated symmetric primitives only, and hardware-accelerated symmetric primitives and Kyber internal operations via *Xkyber*. Note that the leftmost column in each plot in Figure 17 corresponds to the left Kyber-90s columns in Figure 16, while the center columns corresponds to the right Kyber-90s columns. Energy

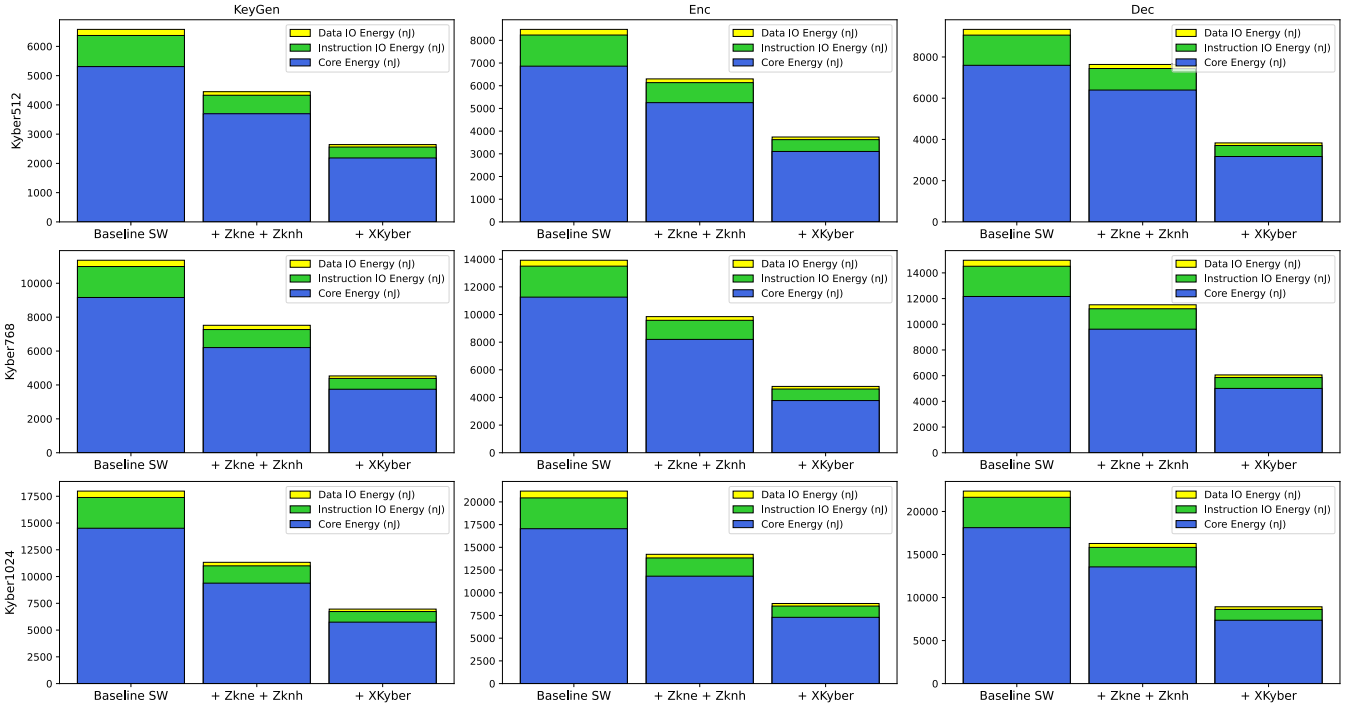



**FIGURE 17.** Energy consumption of Xkyber extension in Kyber-90s (note the scale difference for each plot) for each of the KeyGen, Enc, Dec steps (horizontally) in the Kyber-[512, 768, 1024] security levels (vertically).

consumption gains obtained from the *Zkne* and *Zknh* ISEs being equivalent to gains obtained from the *Xkyber* extension.

## VII. CONCLUSION AND FUTURE WORK

In this work, the adequate choice of symmetric primitives and the use of Instruction Set Extensions (ISEs) is demonstrated to lead to significant improvements in post-quantum key encapsulation with the Crystals-Kyber [2] algorithm in RISC-V-based resource-constrained embedded systems. Results show that in this context Kyber-90s is considerably superior to the to-be-standardized FIPS-202-based variant, with a software-only implementation outperforming hardware-accelerated Keccak-based Kyber-FIPS202. Considering Kyber-512 encapsulation performance, Kyber-90s is 30% (software) 42% (ISE) faster than Kyber-FIPS202.

In addition to Kyber-90s and Kyber-FIPS202, 2 new parametrizations are evaluated: Kyber-Ascon, using the LWC competition winner Ascon [4]; and Kyber-Turbo, using the recently proposed TurboSHAKE XOFs [7]. For the Kyber-512 security level, compared to Kyber-FIPS202, Kyber-Turbo shows 22% (software) and 24% (ISE) increased encapsulation performance while Kyber-Ascon (in the context of a practical cryptosystem) shows 9% (software) and 10% (ISE) better memory footprint, with room for improvements.

Different Kyber parametrizations allow for exploring performance-memory and energy-memory tradeoffs where one is more important than the other. Kyber-Ascon could be chosen for applications where memory usage is the foremost concern, and Kyber-90s where performance or energy are more important. Kyber-90s uses AES, which naturally lends itself to a more coarse-grained ISE implementation than other algorithms used as primitives within Kyber, leading to better results due to ISEs. This brings additional motivation to explore novel Kyber parametrizations, as different algorithms benefit more from hardware acceleration than others.

Additional motivation for exploring additional Kyber parametrizations beyond the to-be-standardized Kyber-FIPS202 can found in exploring resource-sharing at the algorithm level. A practical cryptosystem that requires a KEM like Kyber is very likely to use the encapsulated shared secret as a key for a symmetric cipher such as AES or Ascon-128. From a memory usage standpoint, its immediately beneficial to reuse the same algorithm for symmetric encryption as a primitive within Kyber, like in the Kyber-90s and Kyber-Ascon parametrizations. This does not incur the memory footprint cost of having to instantiate new primitives within Kyber that may not be used anywhere else, as done in the Kyber-FIPS202 and Kyber-Turbo parametrizations. The same rationale can be applied to the hardware acceleration of symmetric primitives: ISEs used to accelerate a given algorithm can have their die area cost diluted if used not only separately, but within Kyber as well. Naturally, performance and energy consumption benefits due to ISEs transitively apply to Kyber as well as the symmetric algorithm used in a standalone manner.

Further investigation of hardware acceleration is performed with a novel *Xkyber* ISE in Kyber-specific operations. Considering Kyber-512 encapsulation, hardware acceleration of the symmetric primitives in Kyber-90s show average performance and energy gains of 42% and 37%, respectively.

Combining hardware acceleration of Kyber-specific operations via *Xkyber* and of the symmetric primitives in Kyber-90s, further gains of 47% and 41% in performance and energy consumption are observed (for Kyber-512 encapsulation), while also reducing Kyber code size by 15%. *Xkyber* area costs are of 13% of the baseline Ibex processor with no ISEs or 4.3K equivalent gates.

For future work, symmetric primitives in Kyber outside of those standardized by NIST can also be investigated. The Grostl [42] and Haraka [43] hash algorithms can benefit from AES hardware acceleration, which can further improve Kyber speeds beyond those measured for Kyber-90s while having a smaller memory footprint due to the exclusion of the costly SHA-256 and SHA-512 round constants.

Additionally, digital signatures also need to be investigated. One option is to use KEM-based authentication [44] for applications in which messages can be sent back and forth instead of traditional digital signatures. This approach enables reuse of hardware accelerated Kyber (where again the idea of resource-sharing at the algorithm level is relevant), but requires multi-round-trip communication, which is not suitable for all applications, such as authenticating binaries stored in non-volatile memory, for which a traditional digital signature must be used.

The NIST PQC competition also comprises digital signatures, for which 3 algorithms have been chosen for standardization: Dilithium [45], Falcon [46] and SPHINCS [47]. Dilithium is closely related to Kyber, where the some of the techniques for Kyber optimizations described in this work could still be applied in Dilithium, but would likely lead to smaller gains than in the Kyber case. The SIMD elements in the *Xkyber* extension stem from the fact that Kyber polynomial coefficients are 12 bits wide, where 2 coefficients can fit in a single word of a 32 bit processor. Dilithium polynomial coefficients are 23 bits wide, where only a single coefficient can fit in a word. Larger bit widths for intermediary results in the Barrett reduction would need to be accommodated into the data path and Ibex temporary registers, which would lead to a worse performing ISE at a higher area cost.

Falcon has the smaller public key and signature size, which at a first glance might be make it the most attractive option for digitalsignatures ina resource-constrained embedded context. However, it requires the use of constant time floating-point operations, which is hard to implement in software-emulated floating point operations, discouraging its use in the context in question, seeing as hardware floating point operations are unlikelyto be available. Finally, SPHINCS shows the biggest signature size. For the aforementioned binary authentication from non-volatile memory example, this would not be significant, seeing as non-volatile memory usage is not a constraint in most applications. Being a hash-based scheme, itsperformanceisheavily reliant on the underlying hash functions. In this case, its integration with hardware accelerated Haraka and use alongside KEM-based authentication seems particularly promising.

## REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, Jan. 1999, doi: 10.1137/s0036144598347011.
[2] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehl. (2022). *CRYSTALS-Kyber-Submission to Round 3 of the NIST Post-Quantum Project*. [Online]. Available: https://pq-crystals.org/kyber/resources.shtml
[3] *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Standard FIPS 203 (IPD), 2023, doi: 10.6028/NIST.FIPS.203.ipd.
[4] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *J. Cryptol.*, vol. 34, no. 3, p. 33, Jul. 2021, doi: 10.1007/s00145-021-09398-9.
[5] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Commun. ACM*, vol. 63, no. 7, pp. 48–57, Jun. 2020, doi: 10.1145/3361682.
[6] lowRISC. (2018). *Ibex RISC-V Core*. [Online]. Available: https://github.com/lowRISC/ibex
[7] G. Bertoni, J. Daemen, S. Hoffert, M. Peeters, G. V. Assche, R. V. Keer, and B. Viguier. (2023). *TurboSHAKE*. Cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2023/342
[8] G. Bertoni, J. Daemen, M. Peeters, G. van Assche, and R. van Keer, "Keccak implementation overview, version 3.2," Keccak Team, Tech. Rep., 2012. [Online]. Available: https://keccak.team/files/Keccak-implementation-3.2.pdf
[9] K. Stoffelen, *Efficient Cryptography on the RISC-V Architecture*. Berlin, Germany: Springer, 2019, pp. 323–340, doi: 10.1007/978-3-030-30530-7_16.
[10] F. Campos, L. Jellema, M. Lemmen, L. Müller, A. Sprenkels, and B. Viguier, *Assembly or Optimized C for Lightweight Cryptography on RISC-V*. Berlin, Germany: Springer, 2020, pp. 526–545, doi: 10.1007/978-3-030-65411-5_26.
[11] M.-J. O. Saarinen. (2022). *Rvkrypto-Fips*. [Online]. Available: https://github.com/rvkrypto/rvkrypto-fips
[12] G. Nisanci, P. G. Flikkema, and T. Yalçin, "Symmetric cryptography on RISC-V: Performance evaluation of standardized algorithms," *Cryptography*, vol. 6, no. 3, p. 41, Aug. 2022, doi: 10.3390/cryptography6030041.
[13] B. Marshall, D. Page, and T. Pham, "Implementing the draft RISC-V scalar cryptography extensions," in *Hardware and Architectural Support for Security and Privacy*. ACM, 2021, pp. 1–8, doi: 10.1145/3458903.3458904.
[14] University of Bristol. (2018). *SCARV: A Side-channel Hardened RISC-V Platform*. [Online]. Available: https://github.com/scarv/scarv
[15] H. Cheng, J. Großschädl, B. Marshall, D. Page, and T. Pham, "RISC-V instruction set extensions for lightweight symmetric cryptography," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, pp. 193–237, Nov. 2022, doi: 10.46586/tches.v2023.i1.193-237.
[16] Chips Alliance. (2023). *Rocket Core*. [Online]. Available: https://github.com/chipsalliance/rocket
[17] B. Marshall, G. R. Newell, D. Page, M.-J.-O. Saarinen, and C. Wolf, "The design of scalar AES instruction set extensions for RISC-V," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, pp. 109–136, Dec. 2020, doi: 10.46586/tches.v2021.i1.109-136.
[18] M.-J. O. Saarinen, "A lightweight ISA extension for AES and SM4," 2020, *arXiv:2002.07041*.
[19] C. Gewehr and F. G. Moraes, "Improving the efficiency of cryptography algorithms on resource-constrained embedded systems via RISC-V instruction set extensions," in *Proc. 36th SBC/SBMicro/IEEE/ACM Symp. Integr. Circuits Syst. Design (SBCCI)*, Aug. 2023, pp. 1–6, doi: 10.1109/sbcci60457.2023.10261964.
[20] C. Gewehr, N. Moura, L. Luza, E. Bernardon, N. Calazans, R. Garibotti, and F. G. Moraes, "Hardware acceleration of authenticated encryption with associated data via RISC-V instruction set extensions in low power embedded systems," in *Proc. IEEE 15th Latin Amer. Symp. Circuits Syst. (LASCAS)*, Feb. 2024, pp. 1–5, doi: 10.1109/lascas60203.2024.10506132.
[21] M. R. Albrecht, C. Hanser, A. Hoeller, T. Pöppelmann, F. Virdia, and A. Wallner, "Implementing RLWE-based schemes using an RSA co-processor," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, no. 1, pp. 169–208, 2018, doi: 10.13154/tches.v2019.i1.169-208.
[22] J.-Y. Park, Y.-H. Moon, W. Lee, S.-H. Kim, and K. Sakurai, "A survey of polynomial multiplication with RSA-ECC coprocessors and implementations of NIST PQC Round3 KEM algorithms in Exynos2100," *IEEE Access*, vol. 10, pp. 2546–2563, 2022, doi: 10.1109/ACCESS.2021.3138807.

[23] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2019, no. 4, pp. 17–61, 2019, doi: 10.13154/tches.v2019.i4.17-61.
[24] G. Xin, J. Han, T. Yin, Y. Zhou, J. Yang, X. Cheng, and X. Zeng, "VPQC: A domain-specific vector processor for post-quantum cryptography based on RISC-V architecture," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 8, pp. 2672–2684, Aug. 2020, doi: 10.1109/TCSI.2020.2983185.
[25] T. Fritzmann, U. Sharif, D. Müller-Gritschneder, C. Reinbrecht, U. Schlichtmann, and J. Sepulveda, "Towards reliable and secure post-quantum co-processors based on RISC-V," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1148–1153, doi: 10.23919/DATE.2019.8715173.
[26] T. Fritzmann, G. Sigl, and J. Sepúlveda, "RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, pp. 239–280, Aug. 2020, doi: 10.13154/tches.v2020.i4.239-280.
[27] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, "A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms," *IEEE Access*, vol. 9, pp. 150798–150808, 2021, doi: 10.1109/ACCESS.2021.3126208.
[28] J. Lee, W. Kim, S. Kim, and J.-H. Kim, "Post-quantum cryptography coprocessor for RISC-V CPU core," in *Proc. Int. Conf. Electron., Inf., Commun. (ICEIC)*, Feb. 2022, pp. 1–2, doi: 10.1109/ICEIC54506.2022.9748834.
[29] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri. (2020). *ISA Extensions for Finite Field Arithmetic—Accelerating Kyber and NewHope on RISC-V*. Cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2020/049
[30] E. Karabulut and A. Aysu, "RANTT: A RISC-V architecture extension for the number theoretic transform," in *Proc. 30th Int. Conf. Field-Programmable Log. Appl. (FPL)*, Aug. 2020, pp. 26–32, doi: 10.1109/FPL50879.2020.00016.
[31] K. Miteloudi, J. Bos, O. Bronchain, B. Fay, and J. Renes. (2023). *PQ.V.ALU.E: Post-Quantum RISC-V Custom ALU Extensions on Dilithium and Kyber*. Cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2023/1505
[32] S. Di Matteo, M. L. Gerfo, and S. Saponara, "VLSI design and FPGA implementation of an NTT hardware accelerator for homomorphic SEAL-embedded library," *IEEE Access*, vol. 11, pp. 72498–72508, 2023, doi: 10.1109/ACCESS.2023.3295245.
[33] A. Satriawan, I. Syafalni, R. Mareta, I. Anshori, W. Shalannanda, and A. Barra, "Conceptual review on number theoretic transform and comprehensive review on its implementations," *IEEE Access*, vol. 11, pp. 70288–70316, 2023, doi: 10.1109/ACCESS.2023.3294446.
[34] *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, Standard FIPS 202, 2015, doi: 10.6028/nist.fips.202.
[35] *Secure Hash Standard (SHS)*, Standard FIPS 180-4, 2015, doi: 10.6028/nist.fips.180-4.
[36] *Advanced Encryption Standard (AES)*, Standard FIPS 197-UPD1, 2001. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf
[37] Intel. (2017). *TinyCrypt Cryptographic Library*. [Online]. Available: https://github.com/intel/tinycrypt
[38] P. Barrett, *Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor*. Berlin, Germany: Springer, 1986, pp. 311–323, doi: 10.1007/3-540-47721-7_24.
[39] E. B. Barker and J. M. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2015, doi: 10.6028/NIST.SP.800-90Ar1.
[40] L. Botros, M. J. Kannwischer, and P. Schwabe. (2019). *Memory-Efficient High-Speed Implementation of Kyber on Cortex-M4*. Cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2019/489
[41] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, pp. 1–25, Jun. 2017, doi: 10.1145/3085572.
[42] P. Gauravaram, L. R. Knudsen, K. Matusiewicz, F. Mendel, C. Rechberger, M. Schläffer, and S. S. Thomsen. (2009). *GROSTL—A SHA-3 Candidate*. [Online]. Available: https://perso.uclouvain.be/fstandae/source_codes/hash_atmel/specs/groestl.pdf
[43] S. Kölbl, M. M. Lauridsen, F. Mendel, and C. Rechberger. (2016). *Haraka V2—Efficient Short-Input Hashing for Post-Quantum Applications*. Cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2016/098
[44] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, and N. Sullivan, "KEM-based authentication for TLS 1.3," Internet Eng. Task Force, 2023. [Online]. Available: https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/02
[45] *Module-Lattice-Based Digital Signature Standard*, Standard FIPS 204 (IPD), 2023, doi: 10.6028/NIST.FIPS.204.ipd.
[46] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-Fourier lattice-based compact signatures over NTRU," *Submission NIST's Post-Quantum Cryptogr. Standardization Process*, vol. 36, no. 5, pp. 1–75, 2018. [Online]. Available: https://falcon-sign.info/
[47] *Stateless Hash-Based Digital Signature Standard*, Standard FIPS 205 (IPD), 2023, doi: 10.6028/NIST.FIPS.205.ipd.



**CARLOS GEWEHR** received the B.Sc. degree in computer engineering from the Universidade Federal de Santa Maria (UFSM), in 2021, and the M.S. degree (Hons.) in computer science from the Pontifical Catholic University of Rio Grande do Sul (PUCRS), in 2024. He is currently pursuing the Ph.D. degree with Carnegie Mellon University. He was a Digital Design Engineer with EnSilica, from 2021 to 2024. His research interests include computer architecture, low-power embedded systems, hardware security, and cryptography.



**LUCAS LUZA** is currently pursuing the B.Sc. degree in computer engineering with the Pontifical Catholic University of Rio Grande do Sul (PUCRS). His research interests include embedded systems, computer architecture, and cryptography. He was part of the winning team of the CASS Student Design Competition at ISCAS 2023.



**FERNANDO GEHM MORAES** (Senior Member, IEEE) received the Electrical Engineering and M.Sc. degrees from the Universidade Federal de Rio Grande del Sul (UFRGS), Porto Alegre, Brazil, in 1987 and 1990, respectively, and the Ph.D. degree from the Laboratoire d'Informatique, Robotique et Microélectronique de Montpellier, France, in 1994. He has been a Full Professor with PUCRS, since 2002. He has authored or coauthored 55 peer-refereed journal articles in the field of VLSI design. His primary research interests include microelectronics, security, FPGAs, reconfigurable architectures, NoCs, and MPSoCs.